



Gestión Incidencias

ILERNA ONLINE

SEBASTIÁN OLAYA TAMAYO

SEMESTRE Y CURSO 2S 2025/2026

ENLACE DRIVE:

https://drive.google.com/drive/folders/1OC58gd_J8dEu8Eq59MXXM4wU5QLFi0A_?usp=sharing

Contenido

1. INTRODUCCIÓN	3
1.1. Motivación y contexto del problema	3
1.2. Abstract	4
1.3. Objetivos del Proyecto	4
1.3.1 Objetivo General	4
1.3.2 Objetivos Específicos	4
1.4. Alcance y Límites del Sistema	5
2. ESTADO DEL ARTE	6
2.1. Análisis de GLPI (Gestionnaire Libre de Parc Informatique)	6
2.2. Análisis de Jira Service Management (Atlassian)	7
2.3. Análisis de Spiceworks Help Desk	7
2.4. Análisis de osTicket	8
2.5. Tabla Comparativa del Estado del Arte	8
2.6. Conclusión y Posicionamiento de la Aplicación	9
3. METODOLOGÍA.....	10
3.1. Justificación de la Metodología de Desarrollo	10
3.2. Fases del Ciclo de Vida del Proyecto	10
4. TECNOLOGÍAS Y HERRAMIENTAS.....	11
4.1. Lenguaje de Programación: Java (JDK 8+).....	12
4.2. Persistencia de Datos: SQLite y JDBC	12
4.3. Interfaz Gráfica de Usuario (GUI): Java Swing.....	13
4.4. Entorno de Desarrollo y Herramientas Auxiliares.....	13
5. PLANIFICACIÓN, DIAGNÓSTICO Y CONTEXTO LABORAL	13
5.1. Planificación Temporal y Cronograma (Diagrama de Gantt)	13
5.2. Análisis de Desviaciones.....	16
5.3. Diagnóstico del Proyecto (Análisis DAFO)	17
5.4. Contexto Laboral y Perfiles Profesionales Requeridos	18
6. ANÁLISIS DEL SISTEMA	19
6.1. Especificación de Requisitos Funcionales (RF)	19
6.2. Especificación de Requisitos No Funcionales (RNF)	20
6.3. Especificación Detallada de Casos de Uso	21
6.4. Modelado del Sistema y Diagramas UML	23
6.4.1. Diagrama de Casos de Uso	24
6.4.2. Diagrama Entidad-Relación (Diseño de la Base de Datos)	24
6.4.3. Diagrama de Clases (Arquitectura Lógica)	25

7. DISEÑO E IMPLEMENTACIÓN	26
7.1. Arquitectura Base: Patrón MVC y DAO	26
7.2. Estructura de Paquetes del Proyecto	27
7.3. Diseño de la Interfaz Gráfica (UI/UX)	27
7.4. Implementación y Fragmentos de Código Críticos	29
7.4.1. Clase ConexionDB y Autogeneración (Patrón Singleton)	29
7.4.2. Inserción Segura y Prevención de Inyección SQL	29
8. DESPLIEGUE Y PRUEBAS	31
8.1. Estrategia de Despliegue y Empaquetado	31
8.2. Plan de Pruebas Funcionales (Caja Negra)	31
8.3. Manejo de Excepciones y Prevención de Fallos Catastróficos	33
9. CONCLUSIONES	34
9.1. Consecución de los Objetivos Planteados	34
9.2. Reflexión sobre los Desafíos Técnicos Superados	34
9.3. Valoración Personal y Profesional	34
10. LÍNEAS DE TRABAJO Y VÍAS FUTURAS	35
10.1. Migración a Arquitectura Cliente-Servidor	35
10.2. Módulo de Notificaciones Automatizadas (JavaMail API)	35
10.3. Generación Automática de Informes (JasperReports)	36
10.4. Desarrollo de un Portal Web de Usuario (Spring Boot)	36
11. BIBLIOGRAFÍA Y WEBGRAFÍA	36

1. INTRODUCCIÓN

1.1. Motivación y contexto del problema

En el actual paradigma tecnológico, la transformación digital se ha consolidado como un proceso ineludible para la supervivencia y competitividad de cualquier organización. Sin embargo, existe una brecha digital significativa cuando observamos el tejido empresarial local. Mientras las grandes corporaciones y multinacionales invierten una gran parte de su capital operativo en la implementación de sistemas complejos de gestión de infraestructuras IT (basados en marcos de trabajo como ITIL o ITSM), las pequeñas y medianas empresas (PYMES), los centros educativos locales y los pequeños comercios a menudo carecen de las herramientas más básicas para gobernar su tecnología.

La motivación principal para el desarrollo de este proyecto nace de la observación directa de esta carencia en el entorno laboral real. En la gran mayoría de microempresas, la gestión del inventario de hardware (ordenadores, impresoras, servidores locales) y el registro del soporte técnico diario se realiza mediante metodologías arcaicas y propensas al error humano. Es habitual encontrar oficinas donde las averías se reportan mediante notas de papel (Post-its), correos electrónicos desestructurados enviados a una bandeja genérica, o en el mejor de los casos, a través de hojas de cálculo compartidas que terminan corrompiéndose con el tiempo debido al acceso simultáneo.

Esta falta de un sistema centralizado genera múltiples deficiencias operativas: pérdida del historial de reparaciones, imposibilidad de auditar qué componentes fallan con mayor frecuencia, duplicidad de esfuerzos por parte del personal de mantenimiento y, en definitiva, una pérdida de tiempo y recursos económicos.

Desde una perspectiva académica y profesional, abordar este problema desarrollando una solución de software propia me permite materializar y unificar las competencias adquiridas durante los dos años del Ciclo Formativo de Grado Superior en Desarrollo de Aplicaciones Multiplataforma (DAM). La creación de este sistema exige la aplicación

práctica de conceptos avanzados de programación orientada a objetos (POO), diseño y administración de bases de datos relacionales, abstracción de la capa de acceso a datos mediante patrones de diseño, y la creación de interfaces gráficas (GUI) usables e intuitivas.

1.2. Abstract

This project consists of the comprehensive development of a desktop application explicitly designed for the basic management of computer incidents and IT asset inventory in small to medium-sized enterprises. The application allows system administrators and technicians to register users, catalog hardware equipment, and open support tickets (incidents), while meticulously tracking their current status throughout their lifecycle.

Built using Java as the core object-oriented programming language, Swing for the graphical user interface components, and SQLite for lightweight, embedded, and highly portable data storage, the system strictly follows an MVC (Model-View-Controller) architecture coupled with the DAO (Data Access Object) design pattern. The main objective of this dissertation is to provide a practical, secure, offline, and user-friendly software tool that integrates programming knowledge, relational databases, and interface design to solve real-world, daily administrative problems without the overhead of complex cloud deployments.

1.3. Objetivos del Proyecto

1.3.1 Objetivo General

Desarrollar, documentar e implementar una aplicación de escritorio nativa, intuitiva y autoinstalable que permita gestionar de forma eficiente, segura y centralizada el ciclo de vida de las incidencias técnicas y el inventario de equipos informáticos de una pequeña organización, garantizando la persistencia de la información mediante el uso de tecnologías estándar del sector.

1.3.2 Objetivos Específicos

Para garantizar la consecución del objetivo general, el desarrollo se ha estructurado en torno al cumplimiento de los siguientes hitos técnicos:

- **Diseño de persistencia relacional:** Diseñar e instanciar una base de datos relacional normalizada hasta la tercera forma normal (3FN), garantizando la integridad referencial mediante el uso de claves primarias y foráneas entre usuarios, equipos e incidencias.
- **Módulo de Autenticación y Seguridad:** Implementar un sistema de control de acceso (*Login*) que valide las credenciales contra la base de datos, previniendo el acceso no autorizado y bloqueando intentos de inyección de código SQL.
- **Gestión del Ciclo de Vida del Hardware (CRUD):** Desarrollar la lógica de negocio y las interfaces necesarias para permitir la creación, lectura, actualización y eliminación de los registros correspondientes al equipamiento informático.
- **Trazabilidad de Incidencias:** Construir un módulo transaccional que obligue a vincular cada nuevo ticket de soporte con un equipo existente y con el usuario que registra la incidencia, asegurando que no existan registros huérfanos.
- **Actualización de Estados:** Habilitar rutinas de actualización que permitan a los técnicos modificar el estado de un ticket desde su apertura ("Pendiente") hasta su cierre definitivo ("Resuelta").
- **Portabilidad y Autodespliegue:** Lograr que la aplicación funcione de manera completamente autónoma. El sistema deberá ser capaz de autogenerar su propio esquema de base de datos en el primer arranque, permitiendo su distribución en un único archivo ejecutable .jar sin requerir configuraciones de servidor adicionales por parte del cliente final.

1.4. Alcance y Límites del Sistema

Para asegurar la viabilidad del proyecto dentro del marco temporal académico, es fundamental acotar el alcance del software.

- **Incluido en el alcance:** El sistema cubrirá la gestión de usuarios locales (administradores/técnicos), el inventario básico de hardware categorizado y el ciclo de vida lineal de las incidencias asociadas a dicho hardware. Todo operando en un entorno *offline* (local).
- **Excluido del alcance:** El sistema no contempla, en su versión actual (v1.0), el envío de notificaciones automáticas por correo electrónico (SMTP), la integración con directorios activos externos (LDAP/Active Directory), ni el acceso concurrente masivo mediante arquitectura cliente-servidor a través de internet, dado que está diseñado específicamente para entornos locales y monopuesto.

2. ESTADO DEL ARTE

El "Estado del Arte" comprende el análisis crítico de las investigaciones, desarrollos tecnológicos y soluciones de software existentes en el mercado que abordan la misma problemática que este proyecto. En el ámbito corporativo, la gestión de incidencias tecnológicas y el control de inventario se engloba bajo los conceptos de sistemas *Helpdesk* (Mesa de ayuda) e *IT Asset Management* (Gestión de Activos IT).

Al tratarse de una necesidad transversal a casi cualquier industria, el mercado ofrece un abanico inmenso de soluciones maduras. No obstante, estas herramientas varían drásticamente en su enfoque, arquitectura, coste de licenciamiento y público objetivo. A continuación, se presenta un estudio detallado de cuatro de las plataformas más representativas de la industria actual, evaluando sus fortalezas y debilidades para justificar la necesidad de la aplicación desarrollada en esta memoria.

2.1. Análisis de GLPI (Gestionnaire Libre de Parc Informatique)

GLPI es una de las soluciones de código abierto (Open Source) más potentes, respetadas y utilizadas a nivel mundial para la gestión integral de servicios IT. Está desarrollada en PHP y fundamentada en bases de datos relacionales como MySQL o MariaDB.

- **Funcionalidades principales:** GLPI es un gigante funcional. Permite realizar un inventario automatizado y exhaustivo de hardware (detectando componentes internos como memoria RAM, procesadores, discos duros) y software (control de licencias, fechas de expiración). Su sistema de *ticketing* es extremadamente robusto y cumple con las normativas internacionales del marco ITIL.
- **Puntos fuertes:** Su mayor atractivo es que, al ser de código abierto bajo licencia GNU GPL, su adquisición es gratuita independientemente del número de agentes o equipos a gestionar. Posee una comunidad global que desarrolla miles de extensiones (*plugins*) para adaptarlo a cualquier necesidad.
- **Limitaciones críticas para PYMES:** El gran talón de Aquiles de GLPI para las pequeñas empresas es la complejidad técnica que requiere su puesta en marcha. Su despliegue exige la instalación, configuración y mantenimiento continuo de un servidor web (entorno LAMP/WAMP), así como la fortificación de la base de datos. Para un pequeño negocio sin un departamento de sistemas dedicado, mantener GLPI operativo y seguro supone un esfuerzo técnico desproporcionado respecto a los beneficios obtenidos.

2.2. Análisis de Jira Service Management (Atlassian)

En el polo opuesto del espectro encontramos a Jira Service Management, desarrollada por la multinacional australiana Atlassian. Se ha convertido en el estándar de facto de la industria tecnológica moderna y corporativa, operando casi exclusivamente bajo un modelo SaaS (Software as a Service) alojado en la nube.

- **Funciones principales:** Destaca por la flexibilidad extrema en la creación de flujos de trabajo (*workflows*) visuales. Permite establecer reglas de automatización complejas, gestionar acuerdos de nivel de servicio (SLA) y conectar los tickets de soporte con el código fuente de los desarrolladores.
- **Puntos fuertes:** Cuenta con una interfaz de usuario vanguardista y una usabilidad muy refinada. Su capacidad para integrarse de forma nativa con herramientas de comunicación corporativa como Slack o Microsoft Teams la hace imbatible en entornos colaborativos a gran escala.
- **Limitaciones críticas para PYMES:** El primer obstáculo es el coste económico. Jira opera mediante un modelo de suscripción mensual por agente, lo cual representa un gasto operativo (OPEX) recurrente e inasumible para pequeños comercios. Además, al ser una plataforma tan completa, sufre del síndrome de exceso de funciones (*feature bloat*); un usuario que simplemente desea anotar que "el ordenador de recepción no enciende" se ve abrumado por decenas de campos de priorización, épicas y etiquetas innecesarias en su contexto.

2.3. Análisis de Spiceworks Help Desk

Spiceworks revolucionó el mercado del software de soporte al ofrecer una plataforma de gestión de incidencias y escaneo de red de forma completamente gratuita, utilizando un modelo de negocio basado en la inserción de publicidad orientada a profesionales de IT.

- **Funciones principales:** Proporciona un portal de usuario final, gestión de tickets mediante correo electrónico y utilidades de monitorización básica de la red de área local.
- **Puntos fuertes:** Elimina la barrera económica y ofrece una puesta en marcha considerablemente más amable que la de sistemas como GLPI, apoyándose en una de las comunidades de soporte técnico más grandes de internet.
- **Limitaciones críticas para PYMES:** La inserción constante de *banners* publicitarios en la interfaz de trabajo reduce la inmersión del usuario y proyecta

una imagen menos profesional si la herramienta es visible para otros empleados. Adicionalmente, Spiceworks ha descontinuado recientemente sus instaladores locales para forzar la migración de sus usuarios a su versión *Cloud*. Esto implica ceder los datos del inventario y de los empleados a servidores ubicados en Estados Unidos, generando preocupaciones legítimas sobre el cumplimiento del Reglamento General de Protección de Datos (RGPD) en el entorno europeo.

2.4. Análisis de osTicket

osTicket es otra alternativa histórica de código abierto orientada puramente a la gestión de tickets de soporte, muy habitual en centros educativos y administraciones públicas de menor tamaño.

- **Puntos fuertes:** Es muy ligero, fácil de traducir y altamente eficiente a la hora de transformar correos electrónicos entrantes en tickets de soporte organizados.
- **Limitaciones críticas:** Su desarrollo avanza de forma muy lenta y su interfaz de usuario ha quedado gráficamente obsoleta en comparación con los estándares modernos. Además, carece de un módulo de inventario de equipos integrado, obligando a las empresas a usar osTicket para las averías y un documento Excel paralelo para saber qué equipos poseen.

2.5. Tabla Comparativa del Estado del Arte

Para sintetizar el análisis, se expone la siguiente matriz comparativa evaluando parámetros clave para el escenario de una microempresa local:

Plataforma Analizada	Modelo de Distribución	Coste Económico	Curva de Aprendizaje	Dependencia de Servidor/Nube	Privacidad del Dato (Local/Cloud)
GLPI	Open Source	Gratuito	Muy Alta	Servidor Web y BD requeridos	Control total (si se aloja en local)

Plataforma Analizada	Modelo de Distribución	Coste Económico	Curva de Aprendizaje	Dependencia de Servidor/Nube	Privacidad del Dato (Local/Cloud)
Jira Service Management	SaaS (Nube)	Pago por Agente/Mes	Alta	Dependencia total de Internet	Datos en servidores de terceros
Spiceworks	Freeware / Cloud	Gratis (Con anuncios)	Media	Dependencia total de Internet	Datos en servidores de terceros
osTicket	Open Source	Gratuito	Media	Servidor Web y BD requeridos	Control total (si se aloja en local)
GestiónIncidentes (Este proyecto)	Desktop App (Standalone)	Gratuito	Muy Baja	No requiere servidor ni red	Privacidad absoluta (Offline)

2.6. Conclusión y Posicionamiento de la Aplicación

Tras la revisión de las alternativas descritas, se evidencia una clara polarización en el mercado del software de soporte IT. En un extremo se sitúan los "monstruos" corporativos, extremadamente potentes, pero con barreras de entrada técnicas (GLPI) o económicas (Jira) insalvables para una PYME. En el otro extremo, las soluciones gratuitas migran inexorablemente hacia la nube (Spiceworks), arrebatando a la pequeña empresa el control directo sobre la privacidad de sus datos.

El nicho de mercado y la razón de ser de este proyecto radican en la **simplicidad operativa y la soberanía del dato**. *GestiónIncidentes* se posiciona deliberadamente como una aplicación de escritorio monolítica tradicional. No requiere instalación de servidores Apache, ni motores MySQL, ni conexión a internet. Al empaquetar la base de

datos (SQLite) junto a la lógica gráfica (Java Swing), el software se convierte en una herramienta "Plug and Play". Su propuesta de valor es ofrecer a cualquier comercio de barrio o pequeña oficina un sistema robusto, gratuito y privado que sustituya de inmediato sus notas de papel o archivos Excel, sin requerir conocimientos informáticos para su mantenimiento.

3. METODOLOGÍA

3.1. Justificación de la Metodología de Desarrollo

La selección de una metodología de desarrollo de software adecuada es el pilar fundamental que garantiza la correcta evolución del código, el cumplimiento de los plazos y la mitigación de riesgos técnicos. Históricamente, la industria del software se apoyaba en el modelo en cascada (*Waterfall*), el cual exige completar y congelar una fase (por ejemplo, el diseño) antes de escribir la primera línea de código.

Dadas las características de este proyecto (desarrollado por un único programador, con un límite temporal estricto impuesto por el calendario académico y la necesidad de adaptar el diseño sobre la marcha), la aplicación de un modelo en cascada tradicional resultaría excesivamente rígido y contraproducente. Por lo tanto, se ha optado por la adopción de una **Metodología Iterativa e Incremental**, fundamentada en los principios de los marcos de trabajo Ágiles (*Agile*).

Esta metodología permite fragmentar la aplicación en módulos funcionales pequeños y manejables (incrementos). De este modo, en lugar de intentar programar todo el sistema de una vez, el desarrollo se enfoca en entregar piezas de software funcionales de forma continua. Para la gestión visual del flujo de trabajo, se ha implementado la filosofía **Kanban**, clasificando las tareas en tres estados básicos: "Por hacer" (*To Do*), "En progreso" (*In Progress*) y "Completado" (*Done*).

3.2. Fases del Ciclo de Vida del Proyecto

Aplicando la metodología iterativa descrita, el ciclo de vida de la aplicación *GestiónIncidencias* se ha estructurado en las siguientes fases secuenciales, pero con capacidad de retroalimentación:

1. **Fase de Elicitación y Análisis de Requisitos:** En esta etapa embrionaria se definió el problema central a resolver. Se acotó el alcance para evitar el conocido *scope creep* (crecimiento descontrolado de las características

del proyecto). Se determinó que la aplicación debía gestionar tres entidades fundamentales: Usuarios, Equipos e Incidencias, estableciendo los límites del Producto Mínimo Viable (MVP).

2. **Fase de Diseño Arquitectónico y Modelado:** Previo a la codificación, se estableció la arquitectura lógica del sistema basada en el patrón Modelo-Vista-Controlador (MVC) y Data Access Object (DAO). Paralelamente, se diseñó el modelo relacional de la base de datos hasta alcanzar la Tercera Forma Normal (3FN), garantizando la ausencia de redundancias y la integridad referencial. Además, se plantearon bocetos (*wireframes*) para definir la experiencia de usuario (UX) en las interfaces.
3. **Fase de Desarrollo e Implementación (Sprints):** Ha sido la fase de mayor carga lectiva, dividida en tres grandes bloques iterativos:
 - *Iteración 1 (Backend Core):* Establecimiento de la conexión con SQLite, desarrollo del motor de autogeneración de tablas y validación de las sentencias DDL (Data Definition Language).
 - *Iteración 2 (Lógica de Negocio y DAO):* Mapeo de las tablas relacionales a objetos Java (POJOs) y programación de las sentencias SQL preparadas (CRUD) para asegurar la persistencia de datos y evitar inyecciones de código.
 - *Iteración 3 (Frontend y Vinculación):* Diseño de los formularios en Java Swing, programación de los manejadores de eventos (*ActionListeners*) y conexión de la interfaz gráfica con la capa lógica.
4. **Fase de Pruebas (Testing) y Depuración:** A medida que se completaba cada módulo, se realizaban pruebas unitarias informales y pruebas funcionales de caja negra. Esto incluía forzar excepciones (por ejemplo, introduciendo cadenas de texto nulas o desconectando la base de datos virtualmente) para comprobar la robustez del manejo de errores mediante bloques try-catch.
5. **Fase de Despliegue y Empaquetado:** La fase final consistió en la compilación del código fuente, la integración de librerías externas (driver JDBC) y la generación del artefacto ejecutable final en formato .jar, listo para su distribución.

4. TECNOLOGÍAS Y HERRAMIENTAS

Para garantizar la estabilidad, escalabilidad y eficiencia del sistema, se ha seleccionado una pila tecnológica (*Tech Stack*) compuesta por herramientas estandarizadas en la

industria del desarrollo corporativo, las cuales han sido la columna vertebral de los conocimientos adquiridos en el CFGS de Desarrollo de Aplicaciones Multiplataforma.

Para lograr la conexión con la base de datos local, se ha integrado el driver oficial JDBC de SQLite (sqlite-jdbc). Esta librería se encarga de traducir las consultas SQL desde la aplicación Java hacia el archivo físico de la base de datos, garantizando que el sistema sea 100% portable y no dependa de controladores externos instalados en la máquina del cliente

4.1. Lenguaje de Programación: Java (JDK 8+)

El corazón lógico del proyecto ha sido programado íntegramente en Java. La elección de este lenguaje se fundamenta en varios pilares críticos:

- **Paradigma Orientado a Objetos (POO):** Java obliga a utilizar un diseño basado en clases y objetos, lo cual facilita enormemente la encapsulación de datos, la herencia y el polimorfismo, permitiendo un código limpio y altamente reutilizable.
- **Portabilidad Universal:** Gracias al paradigma *"Write Once, Run Anywhere"* (Escribe una vez, ejecuta donde sea), el código compilado en *bytecode* es interpretado por la Máquina Virtual de Java (JVM). Esto garantiza que el ejecutable de la aplicación de incidencias funcionará de forma idéntica en el sistema Windows de la recepción de una empresa, como en el sistema Linux o macOS de su departamento de soporte técnico, sin necesidad de recompilar el código.
- **Gestión de Memoria (Garbage Collector):** A diferencia de lenguajes como C++, Java gestiona automáticamente la liberación de memoria de los objetos no utilizados, reduciendo drásticamente las fugas de memoria (*memory leaks*) en aplicaciones diseñadas para estar abiertas durante toda la jornada laboral.

4.2. Persistencia de Datos: SQLite y JDBC

En el ámbito del almacenamiento persistente, se analizó la posibilidad de utilizar motores cliente-servidor robustos como MySQL o PostgreSQL. Sin embargo, dichos motores exigen la instalación de servicios en segundo plano y apertura de puertos, lo que atenta contra el requisito no funcional de "autodespliegue e independencia".

Por ello, se seleccionó **SQLite**. Se trata de un motor de base de datos relacional transaccional, compatible con las propiedades ACID (Atomicidad, Consistencia, Aislamiento y Durabilidad), pero con una arquitectura integrada (*embedded*). La base

de datos completa (esquemas, tablas, índices y registros) se almacena en un único archivo transversal estándar en el disco duro del usuario (database.db). Para la comunicación entre el código Java y SQLite, se ha utilizado la API **JDBC (Java Database Connectivity)** junto con el *driver* específico sqlite-jdbc.

4.3. Interfaz Gráfica de Usuario (GUI): Java Swing

Para el desarrollo del *Frontend*, se descartaron tecnologías web (HTML/CSS/JS) para mantener la aplicación como un ejecutable monolítico de escritorio. Se optó por **Java Swing**, un *toolkit* gráfico nativo de Java que proporciona componentes ligeros (*lightweight components*). Swing ha permitido diseñar interfaces profesionales mediante el uso de contenedores (JFrame, JPanel), gestores de distribución espacial (*Layout Managers*) y componentes interactivos avanzados como tablas de datos (JTable), pestañas de navegación (JTabbedPane) y cuadros de diálogo modales (JOptionPane).

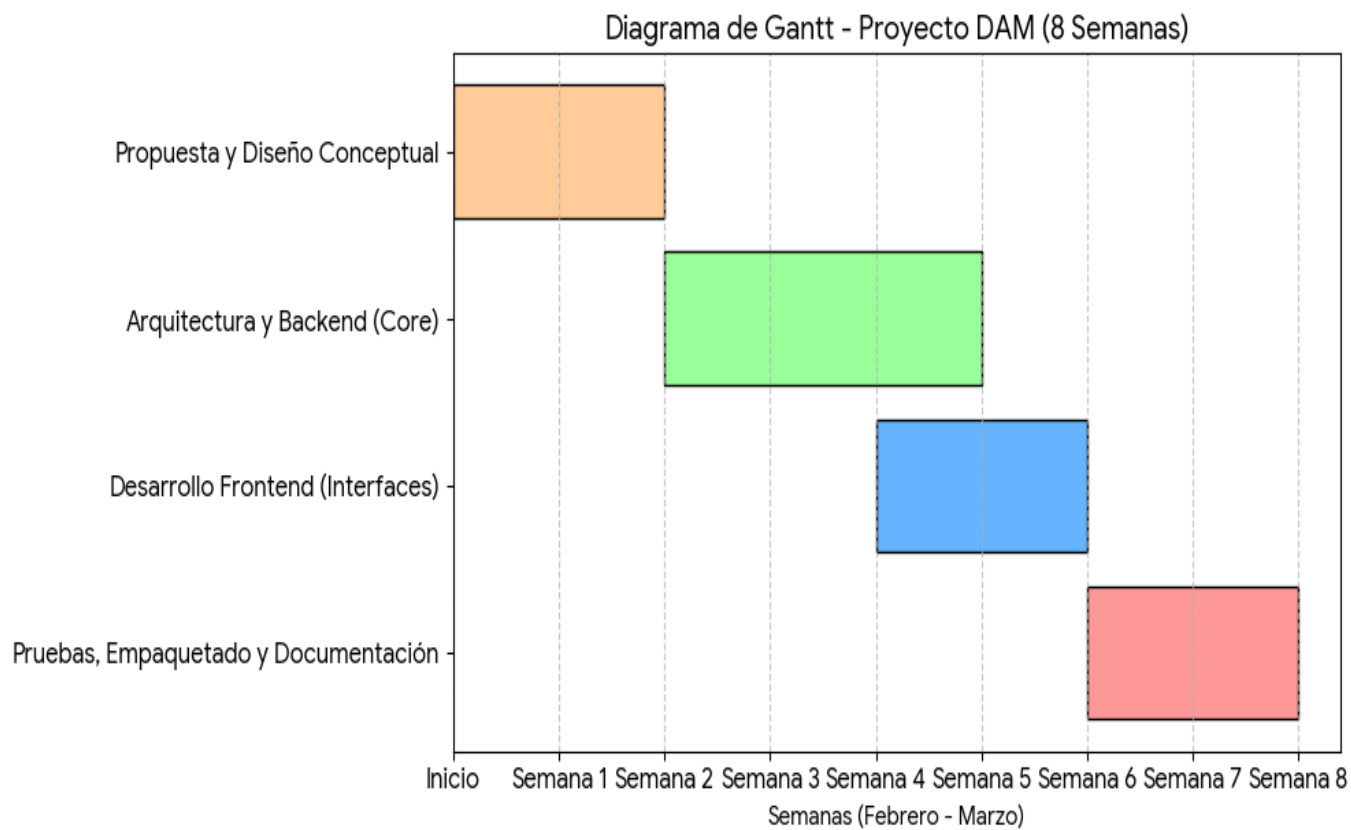
4.4. Entorno de Desarrollo y Herramientas Auxiliares

- **IDE (Entorno de Desarrollo Integrado):** El proyecto ha sido codificado utilizando un IDE moderno (como Eclipse o NetBeans). Estas herramientas aumentan exponencialmente la productividad al proporcionar detección de errores de sintaxis en tiempo real, refactorización automática y un depurador (*debugger*) visual para inspeccionar el estado de las variables en tiempo de ejecución.
- **PlantUML:** Para la ingeniería de software y la generación de la documentación gráfica, se ha recurrido a PlantUML, una herramienta de código abierto que permite dibujar diagramas UML estructurados (Casos de Uso, Entidad-Relación y Clases) a partir de descripciones en texto plano, asegurando una representación arquitectónica fiel a los estándares internacionales.

5. PLANIFICACIÓN, DIAGNÓSTICO Y CONTEXTO LABORAL

5.1. Planificación Temporal y Cronograma (Diagrama de Gantt)

Para asegurar la viabilidad del proyecto dentro del marco de tiempo académico exigido por ILERNA, se trazó un cronograma de trabajo estructurado en 8 semanas. La correcta planificación es vital para evitar el denominado "síndrome del estudiante" y garantizar tiempo suficiente para la resolución de errores imprevistos. A continuación, se detalla la planificación teórica original:



ID	Fases y Tareas Principales	Duración Estimada	Periodo de Ejecución
1	Propuesta y Diseño Conceptual	1 Semana	Semana 1
1.1	Elección de la temática y redacción de la propuesta.	2 días	Semana 1

ID	Fases y Tareas Principales	Duración Estimada	Periodo de Ejecución
1.2	Diseño del Diagrama Entidad-Relación (Base de datos).	3 días	Semana 1
2	Arquitectura y Backend (Core)	2 Semanas	Semanas 2 y 3
2.1	Configuración del proyecto y driver SQLite JDBC.	3 días	Semana 2
2.2	Creación del motor de autogeneración de tablas.	4 días	Semana 2
2.3	Programación de clases Modelo (POJOs) y DAOs básicos.	7 días	Semana 3
3	Desarrollo Frontend (Interfaces)	2 Semanas	Semanas 4 y 5
3.1	Diseño visual del Login y métodos de validación segura.	4 días	Semana 4
3.2	Estructuración del MainFrame (Pestañas y Layouts).	3 días	Semana 4
3.3	Programación del módulo de Equipos (JTable y CRUD).	4 días	Semana 5
3.4	Programación del módulo de Incidencias (Lógica UPDATE).	3 días	Semana 5

ID	Fases y Tareas Principales	Duración Estimada	Periodo de Ejecución
4	Pruebas, Empaquetado y Documentación	3 Semanas	Semanas 6, 7 y 8
4.1	Pruebas funcionales de caja negra y corrección de bugs.	5 días	Semana 6
4.2	Empaquetado del archivo .jar ejecutable.	2 días	Semana 6
4.3	Redacción de la Memoria Técnica	10 días	Semanas 7 y 8
4.4	Preparación de diapositivas y grabación del vídeo de defensa.	4 días	Semana 8

5.2. Análisis de Desviaciones

En el ciclo de vida del desarrollo de software, es inusual que la planificación teórica se cumpla sin alteraciones. Durante la ejecución de este proyecto, se experimentaron las siguientes desviaciones técnicas que obligaron a reajustar los tiempos de las tareas posteriores:

- **Desviación en la Tarea 2.3 (Clases DAO) [+ 4 días]:** Se invirtió más tiempo del estimado en la comprensión y manejo de excepciones propias de SQLite. Se produjo recurrentemente el error SQLITE_BUSY (Database is locked). La solución requirió refactorizar el código para asegurar que todas las conexiones (Connection), sentencias (PreparedStatement) y resultados (ResultSet) se cerraran de forma estricta mediante bloques finally, lo que retrasó el inicio del desarrollo visual.
- **Compensación en la Tarea 3.3 (Módulo de Equipos) [- 4 días]:** El tiempo perdido en la fase de base de datos se logró recuperar durante el diseño de la interfaz gráfica. El descubrimiento y uso de la clase DefaultTableModel facilitó enormemente la carga dinámica de datos en las tablas, evitando tener que

programar renderizadores de celdas personalizados desde cero. Finalmente, el proyecto se alineó nuevamente con la fecha de entrega oficial.

5.3. Diagnóstico del Proyecto (Análisis DAFO)

Para evaluar la madurez, viabilidad y posicionamiento de la aplicación *GestiónIncidencias* en un escenario comercial real, se ha llevado a cabo una auditoría estratégica mediante la matriz DAFO (Debilidades, Amenazas, Fortalezas y Oportunidades).

- **Debilidades (Factores Internos):**

- *Riesgo de hardware local:* Al no poseer sincronización en la nube, si el disco duro físico del ordenador donde reside el archivo `database.db` sufre una falla catastrófica y la empresa no realiza copias de seguridad de forma manual, se perderá la totalidad del historial de incidencias corporativas.
- *Monopuesto:* El uso concurrente está severamente limitado. SQLite no está diseñado para que múltiples técnicos desde diferentes ordenadores escriban incidencias simultáneamente a través de una red local, ya que el archivo se bloquea a nivel de sistema operativo durante las transacciones.

- **Amenazas (Factores Externos):**

- *Saturación del mercado SaaS:* El mercado actual empuja fuertemente hacia la adopción de Software como Servicio. Convencer a una empresa de instalar un software de escritorio tradicional puede generar fricciones si están acostumbrados a acceder a todas sus herramientas mediante un navegador web.

- **Fortalezas (Factores Internos):**

- *Curva de aprendizaje casi nula:* La interfaz, limpia y libre de opciones corporativas avanzadas, permite que cualquier empleado de recepción o técnico junior comience a registrar equipos en menos de cinco minutos de formación.
- *Soberanía absoluta de los datos:* Al ser 100% *offline*, los datos de los usuarios, correos y vulnerabilidades de la empresa no viajan por internet ni se almacenan en servidores extranjeros, garantizando un cumplimiento riguroso de la normativa de protección de datos.

- *Independencia tecnológica*: El empaquetado autónomo permite ejecutar la aplicación sin requerir la presencia física de un administrador de sistemas para configurar puertos, cortafuegos o motores SQL.
- **Oportunidades (Factores Externos):**
 - *Nichos de mercado ignorados*: Existe un vasto sector de pequeñas academias, talleres, asociaciones y microempresas que siguen gestionando sus activos en libretas de papel por miedo a las altas cuotas mensuales de las herramientas *Helpdesk* modernas. Esta herramienta cubre ese vacío ofreciendo un puente gratuito y sencillo hacia la digitalización.

5.4. Contexto Laboral y Perfiles Profesionales Requeridos

Es imprescindible contextualizar que, aunque el presente proyecto se ha planificado y ejecutado de manera unipersonal para cumplir con los criterios de evaluación del ciclo formativo, la construcción de un software de características similares en el tejido empresarial real (por ejemplo, en una consultora tecnológica) habría demandado la colaboración orquestada de diversos perfiles profesionales altamente especializados.

Los roles que habrían intervenido en un entorno de desarrollo corporativo son:

1. **Product Manager / Analista Funcional**: Profesional responsable de la toma inicial de contacto con los clientes o el departamento de soporte técnico. Su labor sería traducir las necesidades difusas del usuario ("queremos saber qué ordenadores fallan más") en historias de usuario técnicas y requisitos funcionales concretos para el equipo de desarrollo.
2. **Administrador de Bases de Datos (DBA)**: Ingeniero enfocado exclusivamente en el diseño del modelo Entidad-Relación y en la optimización de las consultas SQL. Se encargaría de definir índices en la tabla de incidencias para asegurar que, aun con diez mil registros históricos, las búsquedas por "ID de Equipo" devuelvan resultados en milisegundos.
3. **Desarrollador Backend (Java Software Engineer)**: Programador encargado de la lógica "invisible" del sistema. Su responsabilidad principal abarcaría la implementación del patrón DAO, la gestión de la seguridad criptográfica de las contraseñas en el Login y la escritura del código JDBC de conexión a SQLite, garantizando la escalabilidad del sistema.
4. **Desarrollador Frontend (UI/UX Specialist)**: Especialista dedicado a la capa de presentación. Usando Java Swing, su misión sería asegurar que los formularios

sean amigables, que los colores corporativos se respeten y, lo más importante, implementar validaciones preventivas (como que no se pueda pulsar el botón "Resolver" si no hay una incidencia seleccionada en la tabla).

5. **Ingeniero de Calidad (QA Tester):** Perfil indispensable antes de lanzar el producto al mercado. Su trabajo consistiría en someter la aplicación a pruebas de estrés y escenarios extremos (intentar inyectar sentencias SQL maliciosas en el Login, intentar borrar un equipo que tiene incidencias asociadas para probar la integridad referencial) reportando cualquier fallo (*bug*) de vuelta a los desarrolladores.

El hecho de haber asumido e integrado todos estos roles a lo largo de las fases de planificación, diseño, codificación y pruebas empíricas, consolida un perfil profesional polivalente (Desarrollador *Full-Stack*), otorgando una visión panorámica indispensable de todos los engranajes que conforman el ciclo de vida del software comercial.

6. ANÁLISIS DEL SISTEMA

La fase de análisis es el núcleo conceptual de la ingeniería de software. En esta etapa, el objetivo principal no es definir cómo se va a programar el sistema, sino qué debe hacer exactamente para solucionar el problema planteado en la introducción. Para ello, se ha llevado a cabo una exhaustiva toma y especificación de requisitos, separando las necesidades del negocio en requisitos funcionales (comportamiento) y no funcionales (atributos de calidad), para posteriormente modelar estas interacciones mediante el lenguaje UML.

6.1. Especificación de Requisitos Funcionales (RF)

Los requisitos funcionales detallan las operaciones, procesos y transformaciones de datos que el sistema *GestiónIncidencias* debe ser capaz de realizar. Se han categorizado según los módulos lógicos de la aplicación:

Módulo 1: Seguridad y Control de Acceso

- **RF-1.1 (Pantalla de Autenticación):** El sistema debe arrancar mostrando obligatoriamente un formulario de inicio de sesión que impida el acceso a las funciones principales.

- **RF-1.2 (Validación de Credenciales):** El sistema debe capturar el email y la contraseña introducidos por el usuario y contrastarlos contra los registros encriptados de la tabla Usuarios en la base de datos.
- **RF-1.3 (Gestión de Intentos Fallidos):** Si las credenciales no coinciden, el sistema debe limpiar el campo de la contraseña y mostrar un cuadro de diálogo advirtiendo del error, sin cerrar la aplicación.

Módulo 2: Gestión de Inventario (Equipos)

- **RF-2.1 (Registro de Hardware):** El sistema debe proporcionar un formulario para dar de alta nuevos equipos informáticos, exigiendo un nombre identificativo, un tipo (Portátil, Sobremesa, Servidor) y un estado operativo inicial.
- **RF-2.2 (Visualización Dinámica):** El sistema debe extraer todos los equipos de la base de datos y mostrarlos en formato de cuadrícula (tabla), actualizándose automáticamente tras cada nueva inserción.
- **RF-2.3 (Eliminación Segura):** El sistema debe permitir al técnico seleccionar un equipo obsoleto de la tabla y eliminarlo del sistema.
- **RF-2.4 (Restricción de Borrado):** Por motivos de integridad referencial, el sistema debe impedir la eliminación de un equipo si este tiene incidencias históricas asociadas.

Módulo 3: Gestión de Incidencias (Tickets)

- **RF-3.1 (Apertura de Tickets):** El sistema debe permitir crear una nueva incidencia, obligando al usuario a redactar una descripción del problema.
- **RF-3.2 (Vinculación Obligatoria):** Toda incidencia creada debe quedar vinculada automáticamente al identificador (ID) del equipo que sufre la avería y al identificador del usuario que ha iniciado la sesión.
- **RF-3.3 (Asignación de Estado Inicial):** Al registrar una incidencia, el sistema le asignará imperativamente el estado "Pendiente".
- **RF-3.4 (Cierre de Tickets):** El sistema debe permitir seleccionar una incidencia "Pendiente" en la tabla y actualizar su estado a "Resuelta" mediante un botón de acción, dejando constancia en la base de datos para futuras auditorías.

6.2. Especificación de Requisitos No Funcionales (RNF)

Los requisitos no funcionales establecen los criterios de calidad, rendimiento y restricciones tecnológicas que el software debe cumplir para ser considerado viable en un entorno de producción real.

- **RNF-1 (Portabilidad Extrema):** La aplicación debe estar empaquetada en un formato universal (.jar) y ejecutarse sin requerir permisos de administrador en el sistema operativo anfitrión.
- **RNF-2 (Independencia de Motores SQL):** El sistema no debe requerir la instalación de servicios de bases de datos pesados (como MySQL Server). Toda la persistencia debe manejarse mediante un archivo local .db gestionado por SQLite.
- **RNF-3 (Autogeneración DDL):** En su primera ejecución en un ordenador en blanco, el sistema debe detectar la ausencia de la base de datos e inyectar automáticamente los scripts CREATE TABLE necesarios para funcionar inmediatamente.
- **RNF-4 (Usabilidad e Interfaz):** La interfaz gráfica debe estar diseñada para ser intuitiva para personal no técnico. Se debe priorizar el uso de pestañas para evitar la superposición molesta de múltiples ventanas secundarias.
- **RNF-5 (Prevención de Inyección SQL):** El acceso a datos debe estar blindado contra ataques de inyección mediante el uso estricto de sentencias parametrizadas y precompiladas.

6.3. Especificación Detallada de Casos de Uso

Para garantizar que el código se comporte exactamente como se espera, se ha procedido a desglosar las interacciones críticas en especificaciones de casos de uso paso a paso. Estas plantillas definen el contrato entre el actor (usuario) y la máquina.

Caso de Uso 01: Autenticación en el Sistema (Login)

- **Actor Principal:** Usuario (Técnico / Administrador).
- **Precondiciones:** La base de datos existe y contiene al menos un usuario administrador registrado (por ejemplo, *admin@admin.com*).
- **Flujo Principal (Éxito):**
 1. El sistema muestra la interfaz gráfica de LoginFrame.
 2. El actor introduce su correo electrónico y su contraseña.
 3. El actor hace clic en el botón "Entrar".
 4. El sistema captura los datos y valida que no sean nulos.

5. El sistema envía las credenciales a la capa DAO.
 6. La capa DAO ejecuta la consulta SELECT en SQLite.
 7. El sistema verifica que el registro existe.
 8. El sistema destruye la ventana de Login y construye el MainFrame (Panel Principal).
- **Flujos Alternativos:**
 - *4a. Campos vacíos:* El sistema aborta la operación y muestra un mensaje "Por favor, rellene todos los campos".
 - *7a. Credenciales inválidas:* El sistema detiene el flujo y lanza un JOptionPane con el texto "Credenciales incorrectas".

Caso de Uso 02: Alta de Nuevo Equipo Informático

- **Actor Principal:** Usuario Autenticado.
- **Precondiciones:** El usuario ha superado el Login y se encuentra en la pestaña "Equipos".
- **Flujo Principal (Éxito):**
 1. El actor escribe el nombre del equipo (Ej: "PC-Recepción") en el campo de texto.
 2. El actor despliega el menú de "Tipo" y selecciona una opción.
 3. El actor despliega el menú de "Estado" y selecciona una opción.
 4. El actor pulsa el botón "Añadir Equipo".
 5. El sistema extrae los textos de los tres componentes gráficos.
 6. El sistema instancia un objeto de la clase Equipo.
 7. El sistema llama al método registrarEquipo(Equipo e) de la clase EquipoDAO.
 8. El sistema ejecuta la sentencia INSERT INTO Equipos....
 9. El sistema limpia los campos de texto del formulario.
 10. El sistema ordena a la JTable que vuelva a consultar la base de datos para refrescar la vista, mostrando el nuevo equipo.

Caso de Uso 03: Registro de una Nueva Incidencia

- **Actor Principal:** Usuario Autenticado.
- **Precondiciones:** Existe al menos un equipo registrado en el sistema.
- **Flujo Principal (Éxito):**
 1. El actor navega a la pestaña "Incidencias".

2. El actor despliega el JComboBox de equipos, el cual ha sido cargado dinámicamente desde la base de datos, y selecciona el equipo afectado.
3. El actor redacta el problema en el área de descripción (Ej: "La pantalla parpadea").
4. El actor pulsa "Registrar Incidencia".
5. El sistema extrae el identificador oculto (ID) del equipo seleccionado en el menú desplegable.
6. El sistema recupera el ID del usuario actualmente logueado.
7. El sistema envía ambos identificadores, junto con la descripción y el estado por defecto ("Pendiente") al modelo DAO.
8. El sistema ejecuta la inserción en SQLite, cruzando las claves foráneas.
9. El sistema actualiza la tabla inferior de incidencias.

Caso de Uso 04: Cambio de Estado (Resolver Ticket)

- **Actor Principal:** Usuario Autenticado.
- **Precondiciones:** Existe una incidencia en estado "Pendiente" visible en la tabla.
- **Flujo Principal (Éxito):**
 1. El actor hace un clic izquierdo sobre una de las filas de la tabla de incidencias.
 2. El actor pulsa el botón "Marcar como Resuelta".
 3. El sistema valida que efectivamente hay una fila seleccionada.
 4. El sistema extrae el valor de la columna 0 (el ID de la incidencia).
 5. El sistema envía este ID al método cambiarEstado() de la capa de acceso a datos.
 6. El sistema ejecuta una instrucción de actualización: UPDATE Incidencias SET estado = 'Resuelta' WHERE id = ?.
 7. El sistema refresca el modelo de la tabla, y la incidencia aparece ahora como solucionada.
- **Flujos Alternativos:**
 - *3a. Ninguna fila seleccionada:* El sistema ignora el clic o muestra una alerta "Seleccione una incidencia primero".

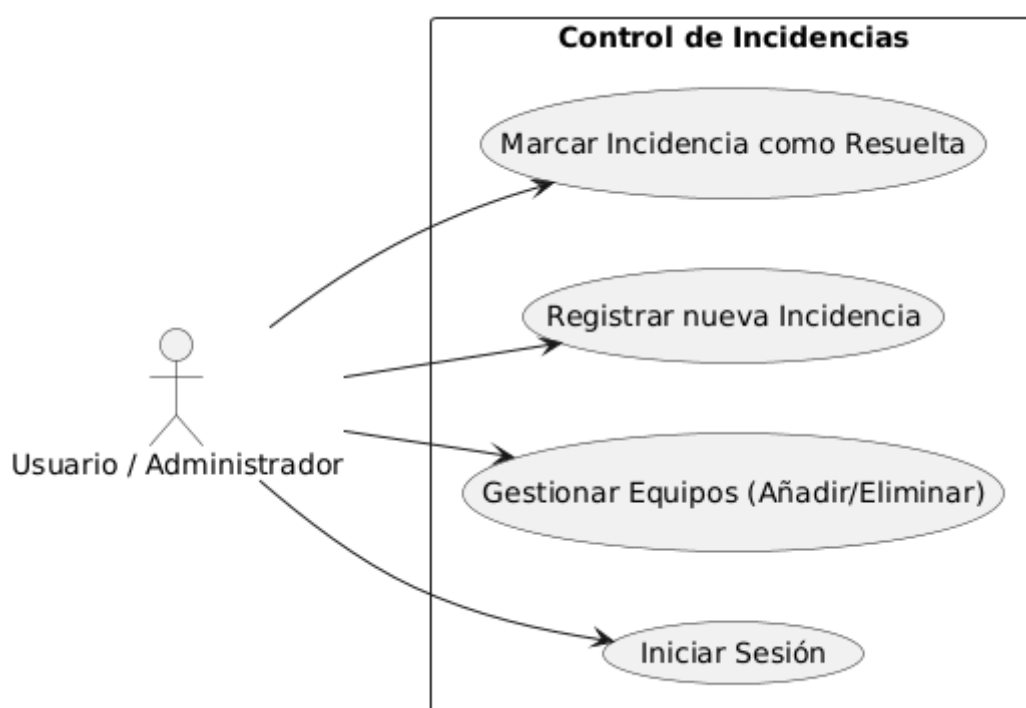
6.4. Modelado del Sistema y Diagramas UML

El Lenguaje Unificado de Modelado (UML) es el estándar industrial para documentar visualmente la arquitectura del software. A partir de los requisitos y casos de uso

previamente analizados, se han diseñado tres diagramas clave que conforman la estructura maestra de la aplicación.

6.4.1. Diagrama de Casos de Uso

Este diagrama ilustra gráficamente las interacciones explicadas en el apartado 6.3. Representa los límites del sistema (el recuadro), al actor principal (el usuario administrador) y las cuatro funcionalidades nucleares a las que tiene acceso. Visualmente, demuestra que el actor debe interactuar con el módulo de sesión antes de poder alcanzar los submódulos de gestión y resolución.

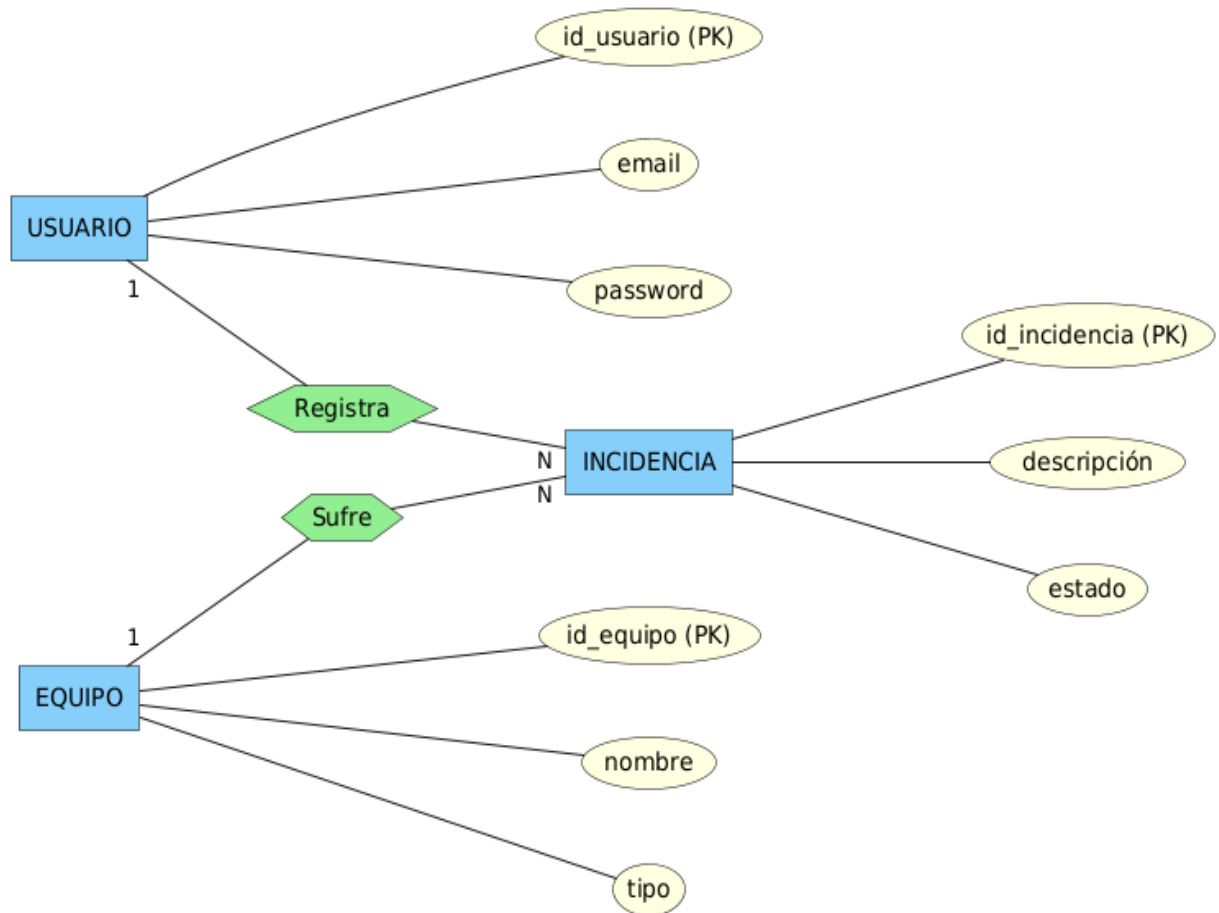


6.4.2. Diagrama Entidad-Relación (Diseño de la Base de Datos)

El modelo relacional es el pilar de la integridad de los datos de este proyecto. El diseño se compone de tres entidades fuertes optimizadas para SQLite:

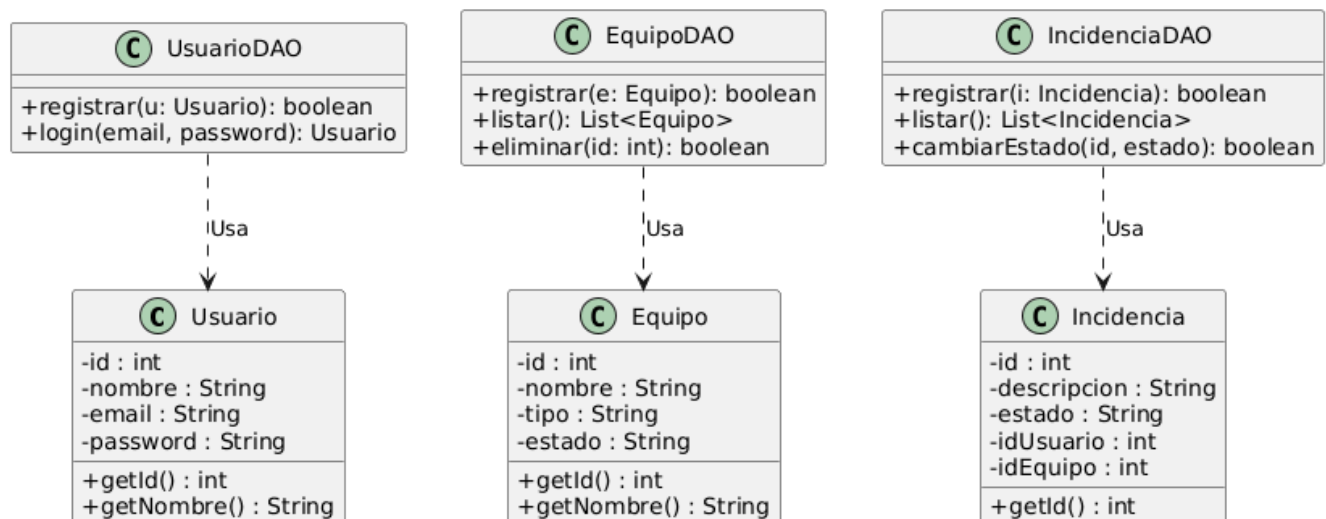
- **Entidad Usuarios:** Encargada de almacenar las credenciales de los trabajadores. Su clave primaria `id` es un campo autoincremental entero.
- **Entidad Equipos:** Actúa como catálogo del hardware físico de la empresa.
- **Entidad Incidencias:** Es la tabla transaccional donde convergen los datos. Su diseño incluye dos claves foráneas (`id_usuario` e `id_equipo`). Esto significa que una incidencia concreta no puede existir en el vacío; está

matemáticamente obligada a pertenecer a un usuario responsable y a un equipo averiado, modelando dos relaciones de cardinalidad uno a muchos (1:N).



6.4.3. Diagrama de Clases (Arquitectura Lógica)

El diagrama de clases representa la traducción de las entidades del mundo real a código Java, siguiendo el principio de responsabilidad única (SOLID) y el patrón DAO. En el esquema se aprecian claramente dos jerarquías. Por un lado, las clases del Modelo (Usuario, Equipo, Incidencia), que son moldes de datos con atributos privados encapsulados. Por otro lado, las clases DAO, que actúan como interfaces lógicas que exponen los métodos CRUD (registrar(), listar(), eliminar()). Las flechas de dependencia punteadas indican que los DAO instancian y utilizan los objetos del modelo para transportar la información desde y hacia la base de datos sin contaminar la interfaz gráfica con código SQL.



7. DISEÑO E IMPLEMENTACIÓN

Este apartado describe cómo se han traducido los requisitos funcionales y los diagramas UML de la fase de análisis a una arquitectura de software tangible. Se detallan las decisiones de diseño a nivel de código, la estructura de paquetes adoptada y se exponen los fragmentos más críticos de la lógica de negocio para justificar la complejidad técnica del desarrollo.

7.1. Arquitectura Base: Patrón MVC y DAO

El sistema se ha construido siguiendo una separación de responsabilidades estricta, huyendo del antipatrón "Spaghetti Code" (donde la interfaz gráfica y las consultas a la base de datos se mezclan en el mismo archivo). Para ello, se ha implementado una arquitectura híbrida basada en el Modelo-Vista-Controlador (MVC) y el Data Access Object (DAO):

- **Capa Modelo (Model):** Compuesta por las clases *Plain Old Java Objects* (POJOs). Estas clases (Usuario, Equipo, Incidencia) son representaciones exactas de las tablas de SQLite en la memoria de la Máquina Virtual de Java. Están fuertemente encapsuladas, exponiendo sus atributos privados únicamente a través de métodos getters y setters.
- **Capa DAO (Data Access):** Es el motor de persistencia. Las clases como EquipoDAO o IncidenciaDAO centralizan y aíslan todo el código SQL. La interfaz gráfica nunca ejecuta un INSERT o un SELECT directamente; en su lugar, pide los datos a la capa DAO. Esta abstracción garantiza que si en el futuro se migra de SQLite a PostgreSQL, la interfaz gráfica no sufrirá ninguna alteración.

- **Capa Vista (View):** Desarrollada con la biblioteca nativa Java Swing. Se encarga exclusivamente de dibujar los componentes en pantalla, capturar los eventos del ratón o teclado mediante ActionListeners y mostrar los resultados al usuario.

7.2. Estructura de Paquetes del Proyecto

Para organizar el código fuente de forma modular dentro del entorno de desarrollo (IDE), se ha jerarquizado el proyecto bajo el dominio raíz `com.gestionincidencias`, dividiéndose en los siguientes subpaquetes lógicos:

- `com.gestionincidencias.main`: Contiene la clase `Main.java`, encargada de inicializar el hilo gráfico (`SwingUtilities.invokeLater`) y lanzar la pantalla de Login.
- `com.gestionincidencias.modelo`: Alberga las clases conceptuales que definen la estructura de los datos.
- `com.gestionincidencias.dao`: Contiene la clase fundamental `ConexionDB.java` y los controladores de persistencia.
- `com.gestionincidencias.vista`: Contiene todos los archivos generados para la interfaz visual (`LoginFrame`, `MainFrame`, `PanelEquipos`, `PanelIncidencias`).

7.3. Diseño de la Interfaz Gráfica (UI/UX)

El diseño de la interfaz ha priorizado la usabilidad (*User Experience*) y la eficiencia para un entorno de oficina. Se descartó el uso de múltiples ventanas flotantes (arquitectura *Multi-Document Interface*), optando por un diseño de ventana única (*Single-Page Application* a nivel de escritorio).

El contenedor principal (`MainFrame`) utiliza un `JTabbedPane` que permite al usuario alternar entre la "Gestión de Equipos" y la "Gestión de Incidencias" mediante pestañas. Para la visualización de los registros masivos, se ha hecho un uso avanzado de componentes `JTable`. Estas tablas están gobernadas por un modelo dinámico (`DefaultTableModel`), lo que permite que cuando la capa DAO inserta un nuevo equipo en la base de datos, la tabla gráfica se refresque automáticamente sin necesidad de que el usuario reinicie la aplicación.

Panel de inicio

Control de incidencias - Login	
Email	
Contraseña	

Panel principal

panel principal - Usuario : Administrador				
Gestión de Equipos		Gestión Incidencias		
Nombre		Tipo		Estado añadir
ID	Nombre	Tipo	Estado	

7.4. Implementación y Fragmentos de Código Críticos

A continuación, se exponen y justifican técnicamente los bloques de código más representativos del núcleo de la aplicación.

7.4.1. Clase *ConexionDB* y Autogeneración (*Patrón Singleton*)

El corazón del sistema *offline* reside en la clase *ConexionDB*. Se ha configurado para utilizar el driver JDBC de SQLite. Además, incluye la inteligencia necesaria para inyectar sentencias DDL (Data Definition Language) si detecta que la base de datos no existe en el sistema anfitrión.

Java

// Fragmento de la clase *ConexionDB.java*

```
public class ConexionDB {
    private static final String URL = "jdbc:sqlite:database.db";
    private static Connection conexion = null;

    public static Connection conectar() {
        try {
            if (conexion == null || conexion.isClosed()) {
                conexion = DriverManager.getConnection(URL);
                crearTablasSiNoExisten(); // Inyección de esquemas
            }
        } catch (SQLException e) {
            System.err.println("Error crítico de conexión: " + e.getMessage());
        }
        return conexion;
    }
}
```

Justificación técnica: Al declarar la variable *conexion* como estática y evaluar *conexion.isClosed()*, se implementa una variación del patrón *Singleton*. Esto evita que la aplicación abra cientos de conexiones simultáneas contra el archivo SQLite, previniendo el temido error *SQLITE_BUSY* (Database is locked) y optimizando el consumo de memoria RAM.

7.4.2. Inserción Segura y Prevención de Inyección SQL

Uno de los requisitos no funcionales primordiales era la seguridad. Si se concatenaran cadenas de texto (Strings) directamente desde los campos de texto del usuario hacia la consulta SQL, el sistema sería vulnerable a Inyecciones SQL. Para impedirlo, la capa DAO utiliza estrictamente la interfaz PreparedStatement.

Java

// Fragmento de IncidenciaDAO.java (Método de inserción)

```
public boolean registrarIncidencia(Incidencia inc) {
    String sql = "INSERT INTO Incidencias (id_equipo, id_usuario, descripcion, estado)
VALUES (?, ?, ?, ?)";

    try (Connection con = ConexionDB.conectar();
        PreparedStatement ps = con.prepareStatement(sql)) {

        // Sanitización y parametrización de variables
        ps.setInt(1, inc.getIdEquipo());
        ps.setInt(2, inc.getIdUsuario());
        ps.setString(3, inc.getDescripcion());
        ps.setString(4, "Pendiente");

        ps.executeUpdate();
        return true;
    } catch (SQLException e) {
        System.err.println("Error al registrar incidencia: " + e.getMessage());
        return false;
    }
}
```

Justificación técnica: El uso del bloque try-with-resources (los paréntesis después del try) es una característica avanzada de Java que garantiza el cierre automático del PreparedStatement y de la Connection inmediatamente después de ejecutar el executeUpdate(), incluso si ocurre una excepción. Los símbolos de interrogación (?) actúan como marcadores de posición, obligando al driver JDBC a precompilar la consulta y tratar los textos del usuario como literales inofensivos, bloqueando cualquier intento de inyección de código.

8. DESPLIEGUE Y PRUEBAS

La fase de validación es crítica para asegurar que el software cumple con los estándares de calidad antes de su entrega y que puede ser desplegado de forma autónoma en el entorno del cliente final.

8.1. Estrategia de Despliegue y Empaquetado

Para convertir el código fuente en un producto distribuible comercialmente, se ha procedido a generar un *Executable JAR Archive*.

- **Gestión de dependencias:** A diferencia de las aplicaciones empresariales gestionadas con Maven o Gradle, este proyecto monolítico integra directamente el archivo `sqlite-jdbc.jar` en su *classpath* de compilación. Durante el proceso de construcción (*build*), el IDE empaqueta las clases compiladas (`.class`), los recursos gráficos (iconos) y el driver de la base de datos dentro de un único fichero denominado `GestionIncidencias.jar`.
- **Requisitos del entorno:** Para ejecutar el sistema, el cliente únicamente necesita disponer de un ordenador con sistema operativo Windows, macOS o Linux, y tener instalada la Máquina Virtual de Java (JRE versión 8 o superior).
- **Ejecución transparente:** Al hacer doble clic sobre el archivo `.jar`, el sistema arranca. La clase `ConexionDB` detecta el entorno y autogenera el fichero `database.db` en el mismo directorio donde reside el ejecutable, cumpliendo así con la promesa de "despliegue cero" (sin instalaciones complejas ni configuraciones de puertos).

8.2. Plan de Pruebas Funcionales (Caja Negra)

Para auditar la robustez de la aplicación y el cumplimiento de los Requisitos Funcionales, se ha ejecutado una extensa batería de pruebas de "Caja Negra". Estas pruebas ignoran el código interno y evalúan el software sometiéndolo a entradas correctas e incorrectas simulando el comportamiento de un usuario real.

NOTA PARA LA EVALUACIÓN

Con el objetivo de facilitar la corrección y verificación de las funcionalidades, se ha incluido en la base de datos un usuario administrador preconfigurado con los siguientes datos de acceso:

- **Usuario:** admin@empresa.com
- **Contraseña:** 1234

A continuación, se documenta la matriz de pruebas críticas ejecutadas sobre la versión candidata a lanzamiento (Release Candidate):

ID	Módulo	Acción Evaluada (Input)	Resultado Esperado	Estado de la Prueba
PR-01	Autenticación	Introducir email registrado y contraseña válida.	El sistema cierra el Login, concede acceso y abre el MainFrame.	ÉXITO
PR-02	Autenticación	Dejar el campo contraseña completamente vacío y pulsar "Entrar".	El sistema bloquea el acceso y muestra un JOptionPane exigiendo datos.	ÉXITO
PR-03	Autenticación	Introducir un email inexistente o contraseña errónea (simulación de intrusión).	Acceso denegado. Se limpian los campos y se muestra alerta "Credenciales incorrectas".	ÉXITO
PR-04	Equipos (CRUD)	Registrar un nuevo equipo con Nombre, Tipo y Estado completos.	Inserción SQL correcta. La tabla visual se refresca instantáneamente mostrando el equipo.	ÉXITO
PR-05	Equipos (CRUD)	Intentar registrar un equipo sin escribir el Nombre.	Operación abortada. Alerta visual de validación de campos obligatorios.	ÉXITO

ID	Módulo	Acción Evaluada (Input)	Resultado Esperado	Estado de la Prueba
PR-06	Equipos (CRUD)	Seleccionar un equipo existente en la tabla y pulsar "Eliminar".	El equipo desaparece de la tabla visual y se elimina del archivo SQLite (DELETE).	ÉXITO
PR-07	Incidencias	Registrar un ticket seleccionando un equipo del desplegable y detallando la avería.	Ticket creado. Se asocia la <i>Foreign Key</i> del equipo y el estado pasa a "Pendiente".	ÉXITO
PR-08	Incidencias	Pulsar "Marcar como Resuelta" sin haber seleccionado ninguna fila en la tabla.	El sistema intercepta el error y avisa al usuario: "Por favor, seleccione una incidencia".	ÉXITO
PR-09	Incidencias	Seleccionar una incidencia "Pendiente" y pulsar "Marcar como Resuelta".	Se ejecuta la sentencia UPDATE. La tabla se refresca mostrando el estado "Resuelta".	ÉXITO
PR-10	Persistencia	Cerrar la aplicación bruscamente, volver a abrirla e iniciar sesión.	Los datos registrados previamente persisten íntegramente en las tablas visuales.	ÉXITO

8.3. Manejo de Excepciones y Prevención de Fallos Catastróficos

Más allá de las pruebas funcionales, el sistema ha sido diseñado de forma defensiva frente a excepciones de sistema. Si el entorno operativo sufre una anomalía (por ejemplo, el usuario cambia los permisos de lectura/escritura de la carpeta impidiendo que SQLite modifique el archivo .db), la aplicación no colapsará (*Crash to Desktop*). En su lugar, el bloque catch (SQLException e) capturará el volcado de pila (*stacktrace*) por consola para el desarrollador, pero emitirá un cuadro de diálogo controlado hacia el usuario final indicando: "Error en la base de datos. Compruebe los permisos del archivo", asegurando así una experiencia de usuario profesional y contenida.

9. CONCLUSIONES

La finalización de este proyecto marca la culminación de un proceso intensivo de aprendizaje, investigación y desarrollo práctico, sirviendo como puente definitivo entre los conocimientos teóricos adquiridos durante el Ciclo Formativo de Grado Superior en Desarrollo de Aplicaciones Multiplataforma (DAM) y las exigencias del mercado laboral real.

9.1. Consecución de los Objetivos Planteados

Al contrastar el producto de software final con los objetivos planteados en el capítulo inicial de esta memoria, se puede afirmar de manera categórica que el objetivo general ha sido alcanzado con éxito. Se ha entregado una aplicación de escritorio plenamente funcional, robusta e intuitiva, capaz de gestionar el ciclo de vida completo de un inventario IT y sus incidencias asociadas. A nivel específico, se han cumplido todos los hitos técnicos propuestos:

- La base de datos relacional (SQLite) asegura la integridad y persistencia de la información.
- El módulo de autenticación protege el acceso al sistema de forma eficaz.
- Las operaciones CRUD (Creación, Lectura, Actualización y Borrado) sobre los equipos operan con total fiabilidad.
- La trazabilidad de los tickets de soporte está garantizada gracias a las restricciones de claves foráneas, evitando la existencia de averías "huérfanas" en el sistema.

9.2. Reflexión sobre los Desafíos Técnicos Superados

El desarrollo no ha estado exento de dificultades técnicas que han puesto a prueba la capacidad de resolución de problemas (problem-solving), una de las habilidades más demandadas en un programador. El mayor reto arquitectónico consistió en el manejo de la concurrencia y los bloqueos del archivo de base de datos (SQLITE_BUSY). Este obstáculo obligó a investigar más allá del temario base, llevándome a comprender a un nivel profundo cómo funciona el *Garbage Collector* de Java y la importancia crítica de cerrar manualmente los flujos de datos (ResultSets y PreparedStatement) mediante bloques try-with-resources. Superar este reto no solo estabilizó la aplicación, sino que mejoró significativamente la calidad y eficiencia del código fuente escrito.

9.3. Valoración Personal y Profesional

A nivel personal, el desarrollo integral de *GestiónIncidencias* me ha permitido experimentar la presión y la responsabilidad inherentes al rol de un desarrollador *Full-Stack*. He tenido que abandonar la comodidad de programar pequeños ejercicios aislados para enfrentarme a la creación de un ecosistema de software completo, donde una decisión de diseño en la base de datos afecta directamente a cómo debe dibujarse un botón en la interfaz gráfica. Esta visión de 360 grados sobre el ciclo de vida del software me otorga la confianza técnica necesaria para incorporarme al sector profesional, sabiendo que soy capaz de transformar una necesidad de negocio abstracta en una herramienta digital concreta y empaquetada.

10. LÍNEAS DE TRABAJO Y VÍAS FUTURAS

El software entregado en su versión 1.0 cumple a la perfección con el Producto Mínimo Viable (MVP) estipulado para una microempresa. Sin embargo, la arquitectura MVC y la encapsulación mediante el patrón DAO han sido diseñadas específicamente para ser escalables. De cara a una futura comercialización o despliegue en organizaciones de mayor tamaño, se proponen las siguientes líneas de ampliación y mejora técnica:

10.1. Migración a Arquitectura Cliente-Servidor

La limitación más evidente de la versión actual es su naturaleza monopuesto (offline). Una evolución natural del proyecto sería sustituir el motor integrado SQLite por un servidor de bases de datos relacional centralizado, como **MySQL o PostgreSQL**. Gracias al patrón DAO implementado, esta migración requeriría únicamente modificar la clase ConexionDB y las cadenas de conexión JDBC, dejando intactas las clases del modelo y la interfaz gráfica. Esto permitiría que decenas de técnicos conectados a la misma red de área local (LAN) pudieran abrir y resolver incidencias simultáneamente, transformando la herramienta en un sistema colaborativo real.

10.2. Módulo de Notificaciones Automatizadas (JavaMail API)

Actualmente, el usuario debe abrir la aplicación y consultar activamente la tabla para comprobar si el estado de una incidencia ha cambiado. Para dotar al sistema de un comportamiento proactivo, se propone la integración de la librería **JavaMail API**. El objetivo es programar *Triggers* (disparadores) lógicos que envíen un correo electrónico automatizado (vía protocolo SMTP) al empleado que reportó la avería en el momento exacto en que un técnico pulse el botón "Marcar como Resuelta", mejorando drásticamente los tiempos de comunicación interna de la empresa.

10.3. Generación Automática de Informes (JasperReports)

La toma de decisiones gerenciales requiere datos cuantificables. Se proyecta la inclusión de un módulo de "Dashboard Analítico" apoyado en la tecnología **JasperReports**. Este módulo permitiría cruzar los datos históricos de la base de datos para generar y exportar informes en formato PDF que respondan a preguntas clave del negocio, tales como:

- ¿Cuál es el tiempo medio de resolución de averías de nuestro departamento?
- ¿Qué equipos informáticos (por ID o por Tipo) sufren la mayor tasa de fallos mensuales y, por tanto, deberían ser sustituidos definitivamente en lugar de reparados?

10.4. Desarrollo de un Portal Web de Usuario (Spring Boot)

Para evitar que cada empleado de la empresa necesite tener instalada la aplicación de escritorio Java nativa, la vía de desarrollo a largo plazo contempla la creación de un portal web complementario. Utilizando el *framework* **Spring Boot** para crear una API RESTful, se podría exponer la base de datos actual hacia una interfaz web ligera (desarrollada en Angular o React). De esta forma, el técnico de soporte seguiría utilizando la potente aplicación de escritorio (Swing) desarrollada en este proyecto, mientras que el resto de los empleados accederían mediante su navegador web exclusivamente para rellenar un formulario rápido y pedir ayuda.

11. BIBLIOGRAFÍA Y WEBGRAFÍA

Para el desarrollo del código, la investigación de mercado y la redacción formal de esta memoria, se han consultado las siguientes fuentes técnicas y documentales:

- **Oracle Corporation (2026). Java Documentation and API Reference.** Fundamental para la implementación de las clases nativas y el manejo de excepciones estructurales. Recuperado de: <https://docs.oracle.com/en/java/>
- **SQLite Consortium (2026). SQLite Official Documentation.** Consultado para la sintaxis exacta de autogeneración de esquemas relacionales y comprensión de los bloqueos de archivo transaccionales. Recuperado de: <https://www.sqlite.org/docs.html>
- **Xerial (2026). SQLite JDBC Driver.** Repositorio oficial del controlador utilizado para conectar la Máquina Virtual de Java con el archivo local .db. Recuperado de: <https://github.com/xerial/sqlite-jdbc>

- PlantUML Team (2026). *PlantUML Language Reference Guide*. Herramienta de código abierto utilizada para la generación estandarizada de todos los diagramas de Casos de Uso, Clases y Entidad-Relación incluidos en el apartado de análisis. Recuperado de: <https://plantuml.com/es/>
- Proyecto GLPI (2026). *Gestionnaire Libre de Parc Informatique*. Herramienta analizada en el apartado "Estado del Arte" como referencia corporativa. Recuperado de: <https://glpi-project.org/>
- Atlassian (2026). *Jira Service Management Features*. Evaluado durante la fase de análisis de mercado y competidores SaaS. Recuperado de: <https://www.atlassian.com/es/software/jira/service-management>
- Materiales Didácticos de ILERNA Online. Temarios correspondientes a los módulos de Programación, Entornos de Desarrollo, Acceso a Datos y Desarrollo de Interfaces del CFGS DAM, que han servido como base teórica fundamental para la estructuración de este proyecto final.

12. ANEXOS

1. Manual de Instalación (Paso a paso)

Este manual explica cómo dejar la aplicación lista para funcionar en un ordenador nuevo.

1. **Requisito previo (Java).** Para que el programa funcione, su ordenador debe tener instalado Java. Si no lo tiene, puede descargarlo gratis desde la web oficial de Java (java.com).



Recursos de ayuda

[Por qué se recomienda Java 8](#)
[¿Qué es Java?](#)
[Eliminar versiones antiguas](#)
[FAQ de Windows](#)
[Seguridad](#)
[Support](#)
[Más ayuda](#)

Esta descarga está destinada a usuarios finales que necesiten Java para ejecutar aplicaciones en equipos de escritorio o ordenadores. Java 8 se integra con su sistema operativo para ejecutar aplicaciones Java instaladas de forma independiente. Si se le ha pedido que ejecute una aplicación de escritorio, lo más probable es que necesite esta versión.

Animamos a los [desarrolladores](#) a descargar la versión más reciente de Java Development Kit desde [Descargas de OTN](#).

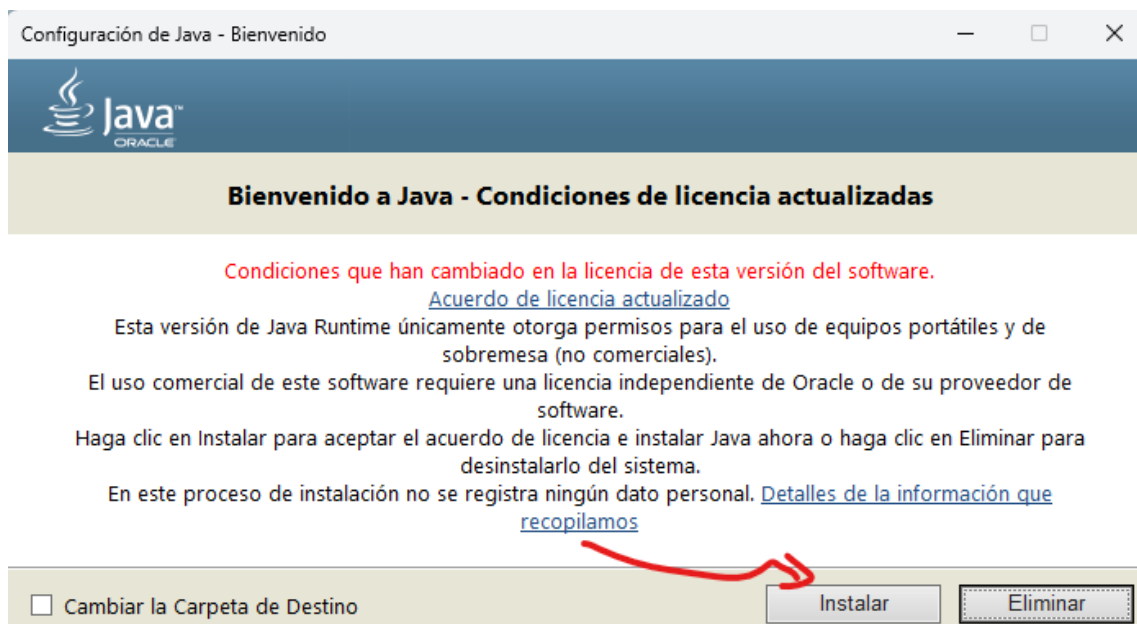
Los usuarios de Enterprise con acceso a [My Oracle Support](#) o a [Oracle Software Delivery Cloud](#) pueden realizar la descarga d

Version 8 Update 481

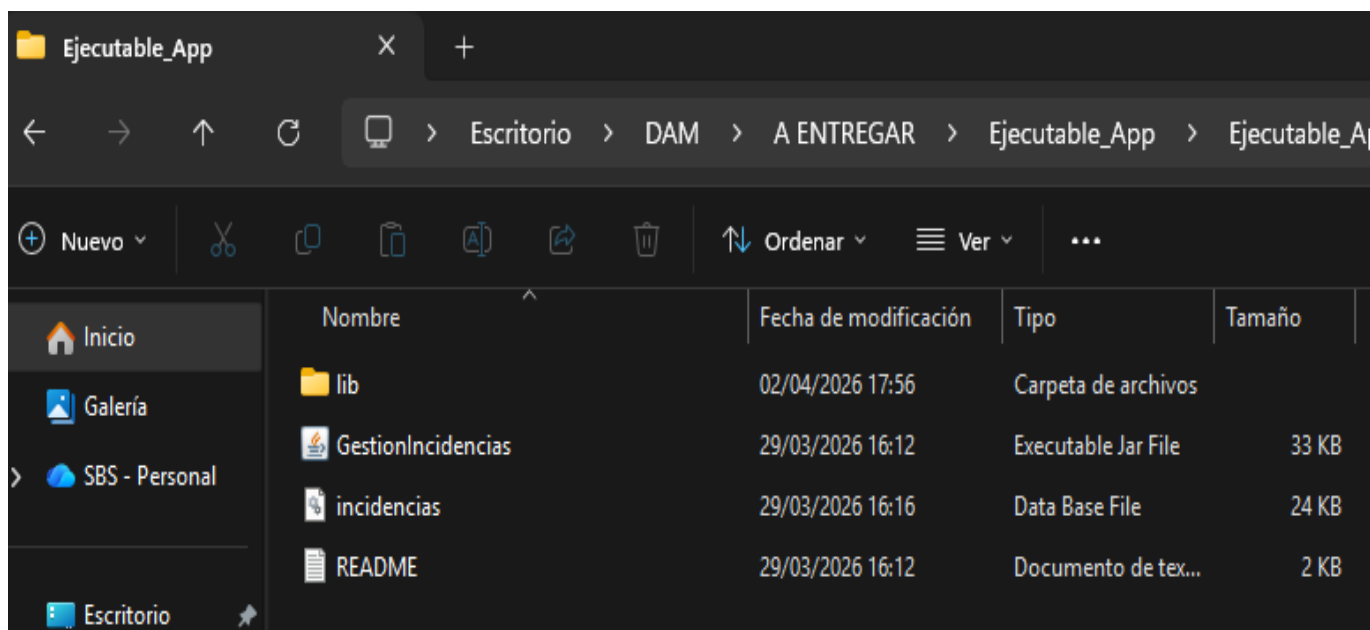
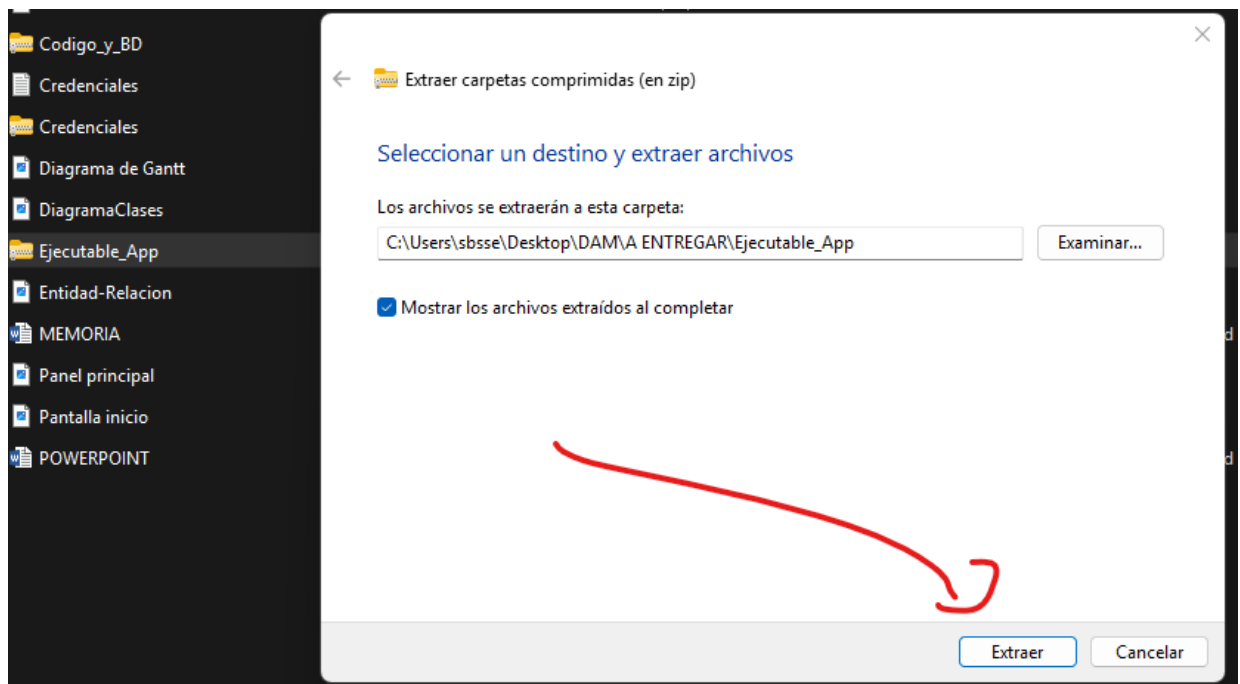
Fecha de publicación: January 20, 2026

Filesize: 38.50 MB

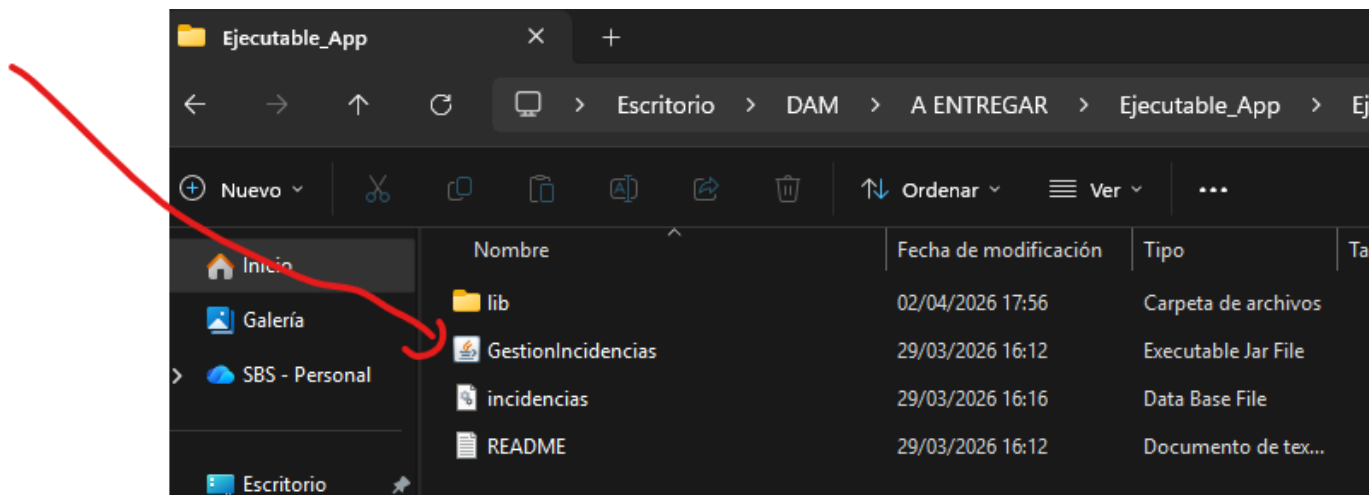
[Descargar Java para escritorios](#)



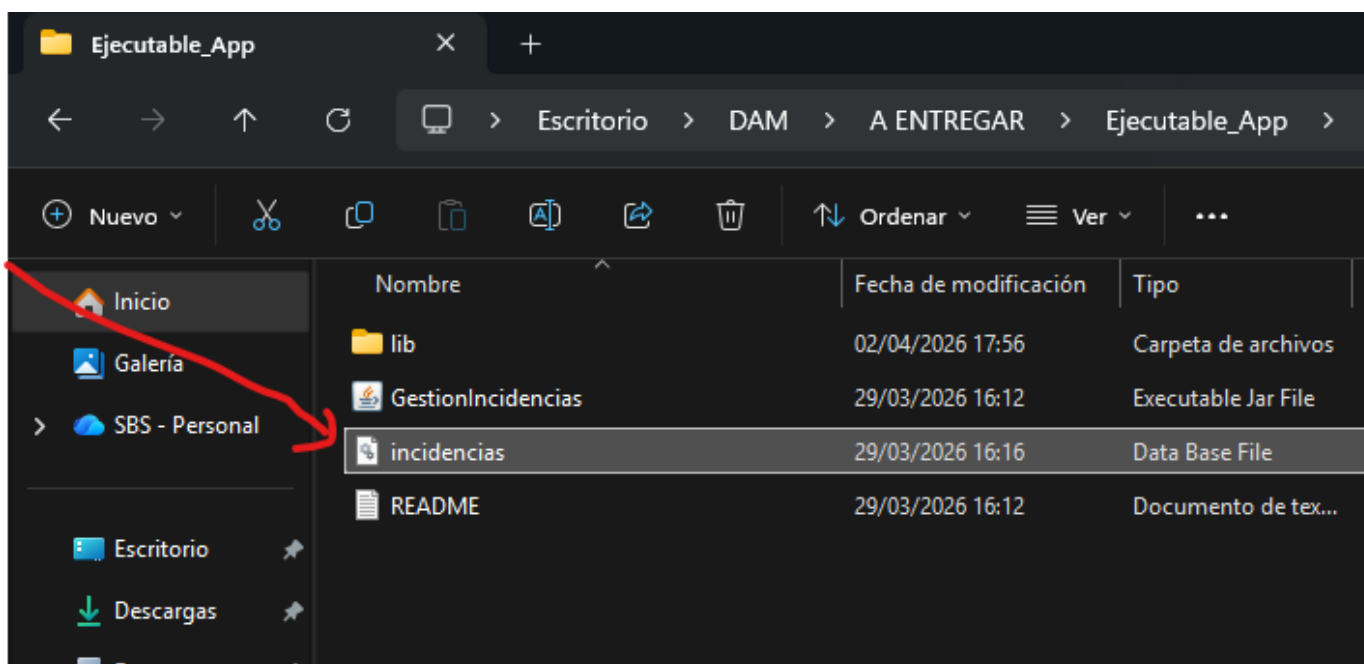
2. **Descarga y ubicación.** Descomprima el archivo ZIP enviado en una carpeta de su ordenador (por ejemplo, en el Escritorio).



3. **Primera ejecución.** No necesita instalar nada. Solo tiene que hacer doble clic sobre el archivo llamado GestionIncidencias.jar.



4. **La Base de Datos.** La primera vez que abra el programa, este creará automáticamente un archivo llamado incidencias en la misma carpeta. No borre este archivo, ya que es donde se guarda toda su información".



2. MANUAL DE USUARIO

Este manual tiene como objetivo guiar al usuario final en el manejo de la aplicación **GestiónIncidencias**, detallando los pasos necesarios para administrar el inventario y los tickets de soporte.

1. Acceso al sistema

Al iniciar la aplicación, se presentará la pantalla de seguridad. Para acceder al panel de gestión, el usuario deberá introducir sus credenciales.

Paso 1: Introduzca el correo electrónico y la contraseña autorizada.

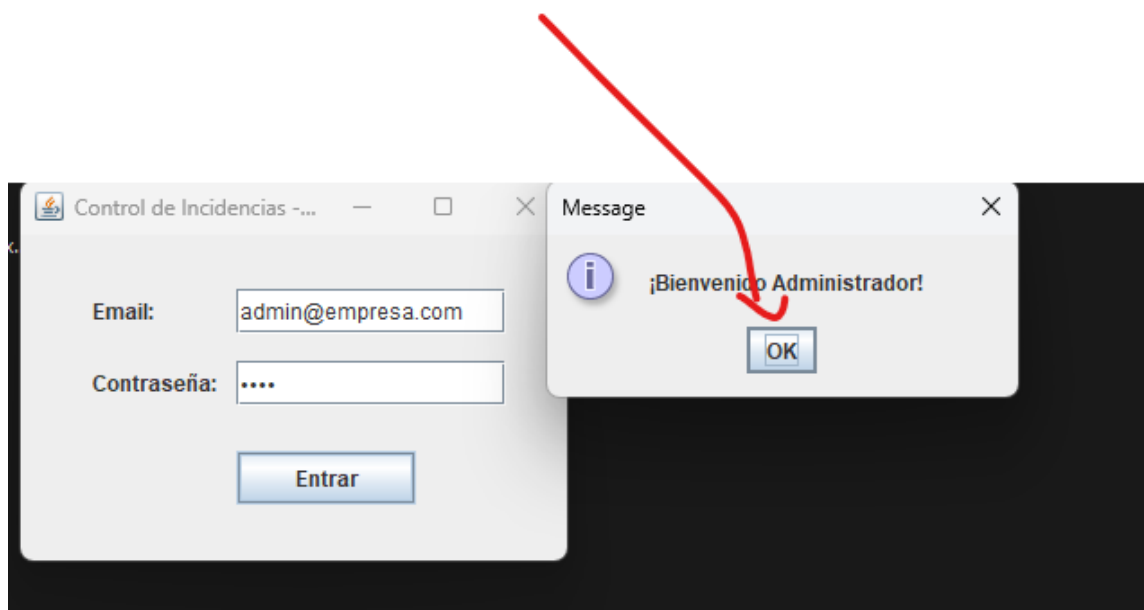
Email: admin@empresa.com

Contraseña: 1234 (Sin espacios)

Paso 2: Pulse el botón "Entrar".

Paso 3: Pulse ok

Nota: Utilice el correo "admin@empresa.com" y la contraseña "1234"



2. Pantalla Principal y Navegación

Una vez dentro, accederá al panel de control. La interfaz está dividida en dos secciones principales accesibles mediante las pestañas superiores:

1. **Pestaña Equipos:** Gestión del inventario físico (PC, Portátiles, Impresoras).
2. **Pestaña Incidencias:** Gestión de averías y reparaciones.

Panel Principal - Usuario: Administrador

Gestión de Equipos | Gestión de Incidencias

Nombre: Tipo: **Portátil** Estado: **Operativo** **Añadir**

ID	Nombre	Tipo	Estado
----	--------	------	--------

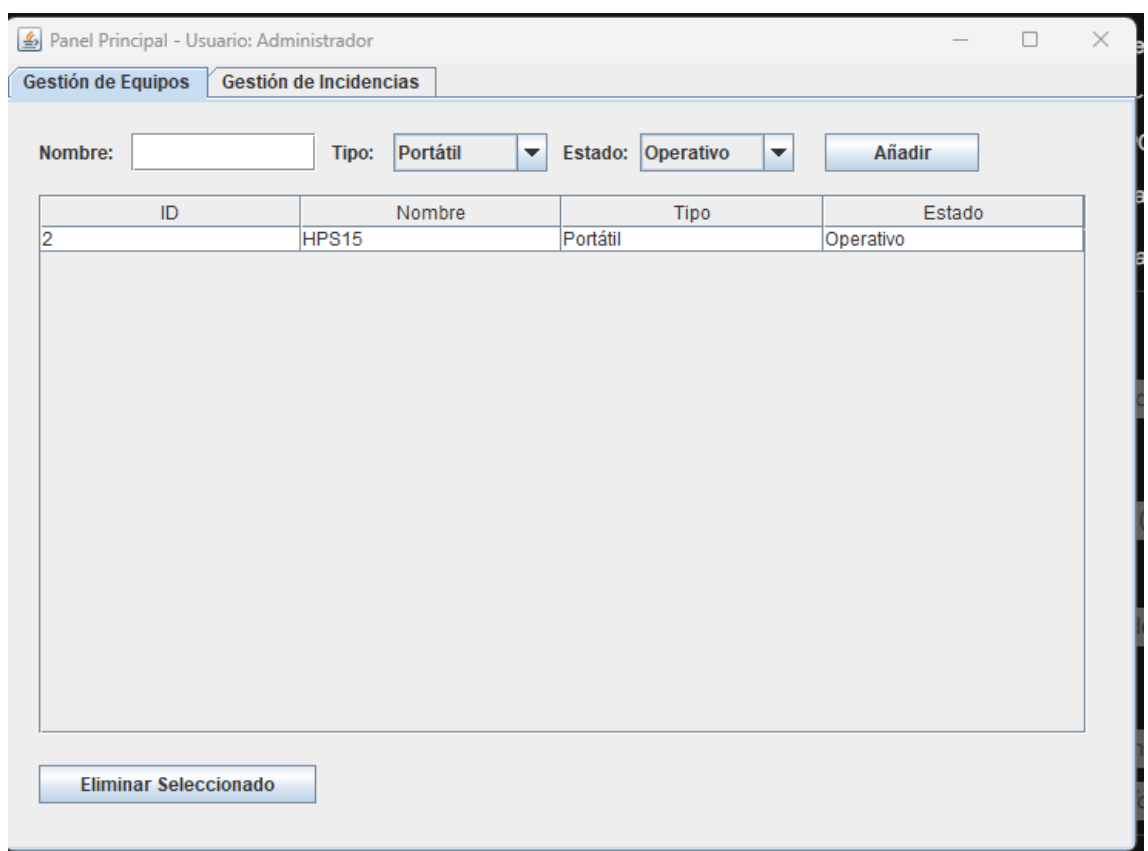
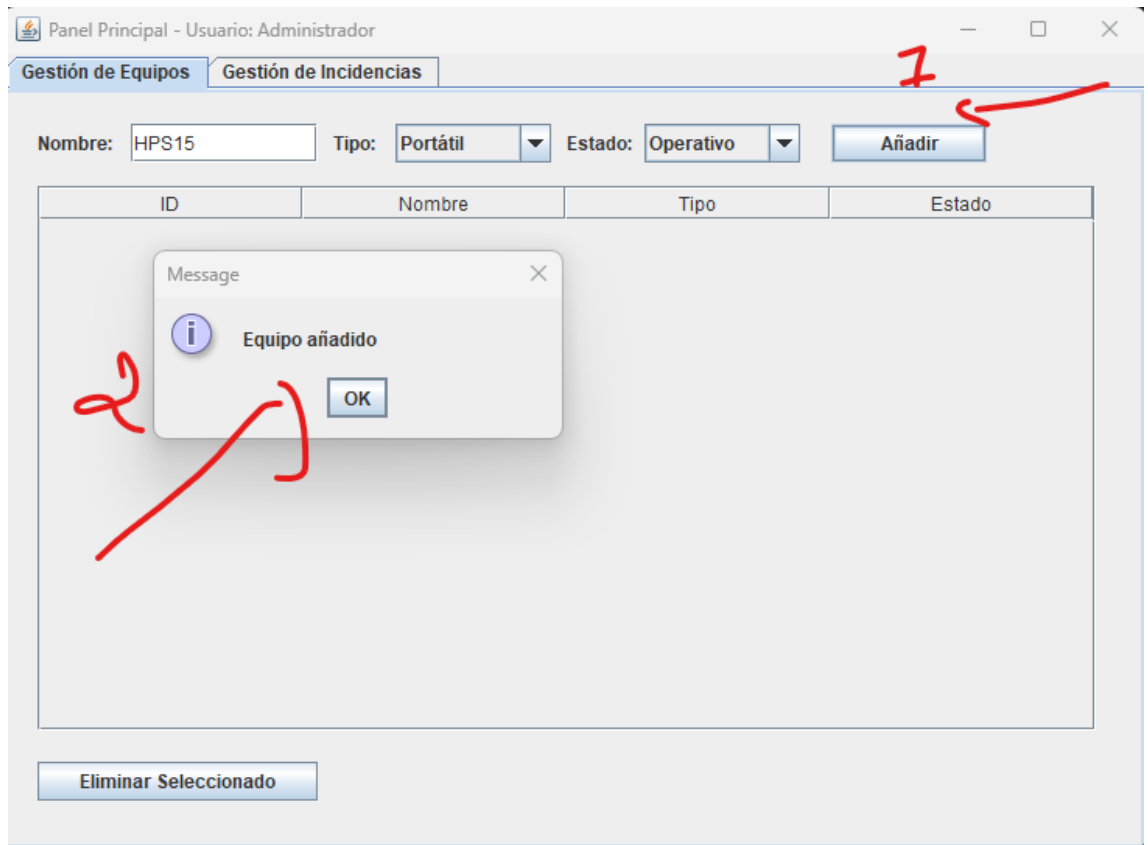
Eliminar Seleccionado

3. Gestión del Inventario (Equipos)

En esta pestaña podrá registrar todos los dispositivos de la empresa.

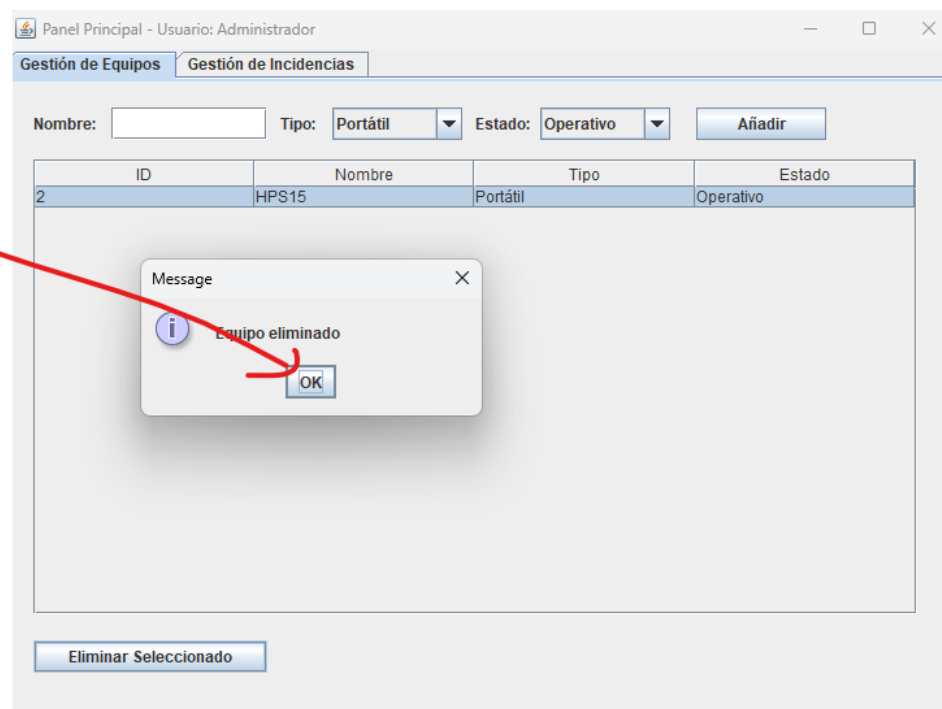
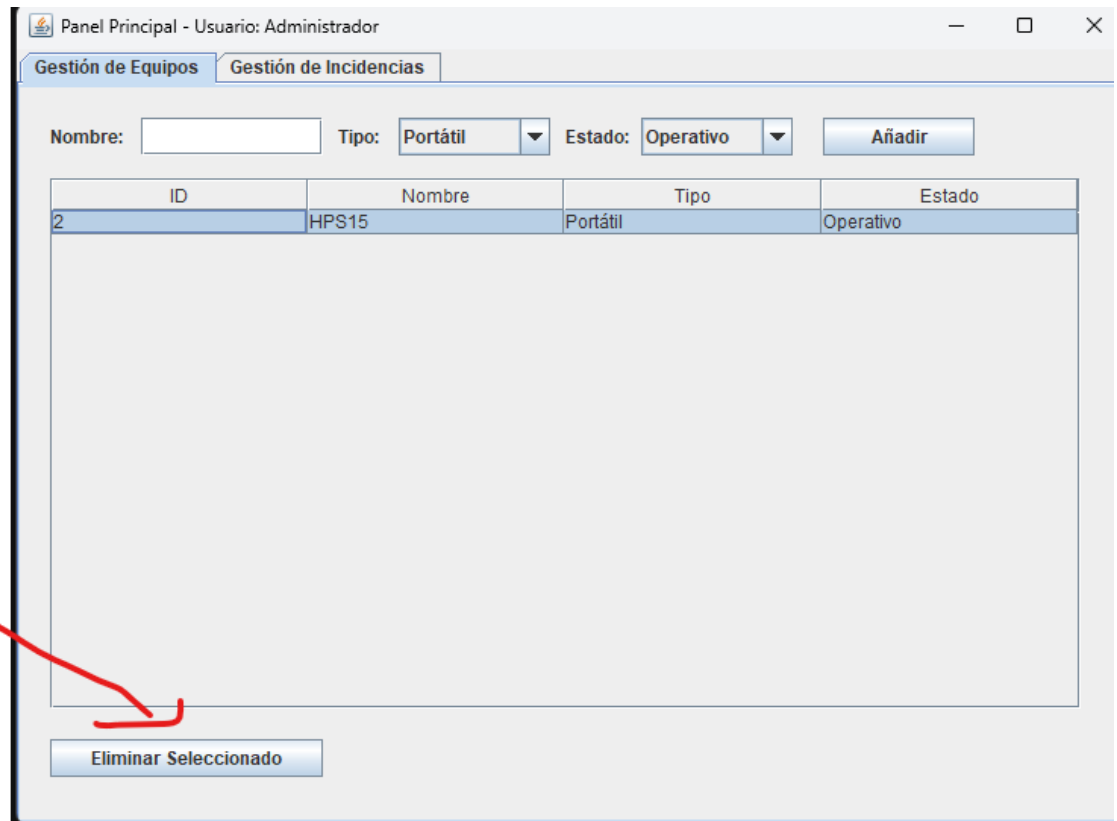
3.1. Añadir un nuevo equipo

1. Rellene los campos del formulario de la izquierda (**Nombre**, **Tipo** y **Estado**).
2. Pulse el botón "**Añadir**".
3. Pulsa el botón "**OK**" cuando nos salga el mensaje de equipo añadido
4. El equipo aparecerá inmediatamente en la tabla de la derecha.



3.2. Eliminar un equipo

Si un equipo se da de baja, selecciónelo en la tabla con un clic y pulse el botón "Eliminar" y después confirmamos la eliminación con "OK"



Automáticamente el equipo desaparece del listado

Panel Principal - Usuario: Administrador

Gestión de Equipos Gestión de Incidencias

Nombre: Tipo: Portátil ▼ Estado: Operativo ▼ Añadir

ID	Nombre	Tipo	Estado
----	--------	------	--------

Eliminar Seleccionado

4. Gestión de Tickets (Incidencias)

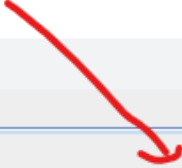
Esta sección permite llevar el control de qué le pasa a cada equipo y si ya se ha arreglado.

4.1. Registrar una nueva incidencia

1. En el menú desplegable, seleccione el **Equipo** que presenta el problema (la lista se carga automáticamente desde su inventario)

Panel Principal - Usuario: Administrador

Gestión de Equipos Gestión de Incidencias

Problema: Equipo: HPS15 (Portátil)  Registrar Incidencia

ID	Descripción	Estado	ID Equipo	ID Usuario
1	no enciende la panta...	Resuelta	1	1

Marcar como Resuelta

2. Escriba una breve **Descripción** del fallo (Ejemplo: "La pantalla no enciende") y Pulse "**Guardar**". La incidencia aparecerá con el estado "**Pendiente**".

Panel Principal - Usuario: Administrador

Gestión de Equipos **Gestión de Incidencias**

Problema: Equipo:

1 2

ID	Descripción	Estado	ID Equipo	ID Usuario
1	no enciende la panta...	Resuelta	1	1

3. Pulse "Ok". La incidencia aparecerá con el estado "Pendiente".

Panel Principal - Usuario: Administrador

Gestión de Equipos **Gestión de Incidencias**

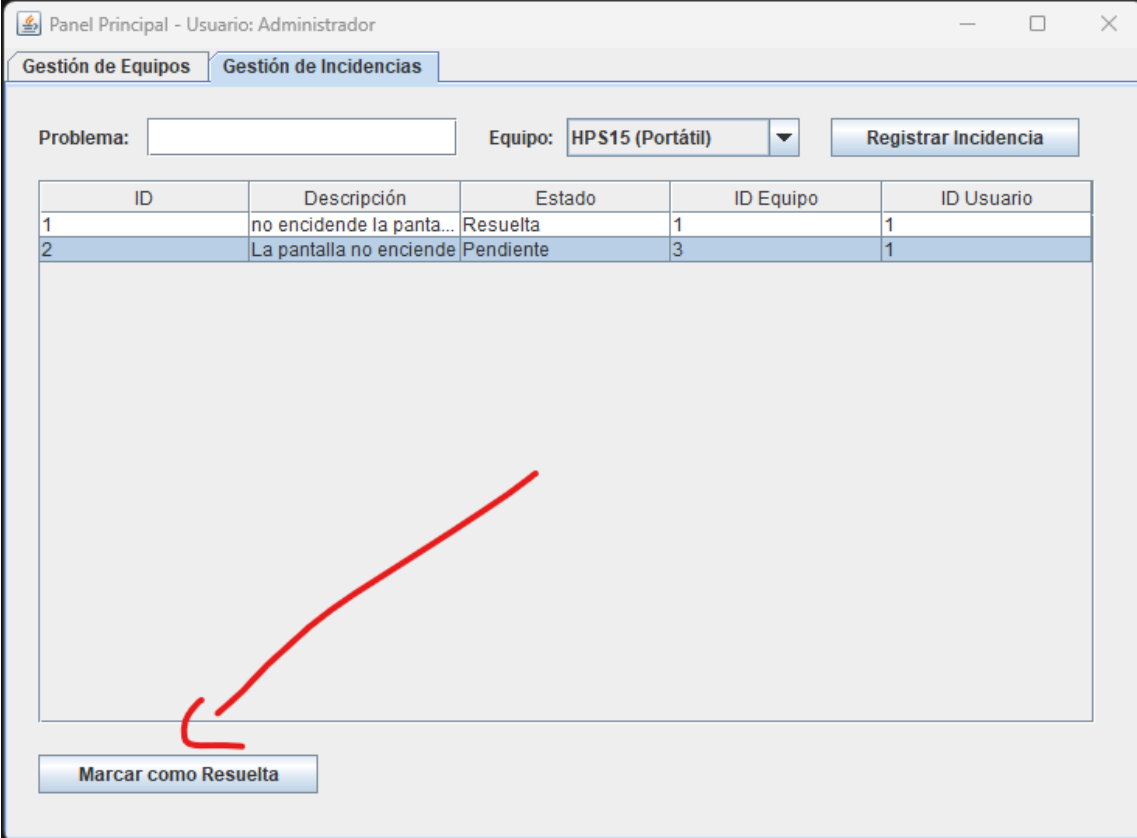
Problema: Equipo:

ID	Descripción	Estado	ID Equipo	ID Usuario
1	no enciende la panta...	Resuelta	1	1
2	la pantalla no enciende	Pendiente	3	1

4.2. Resolver una incidencia

Cuando el problema técnico haya sido solucionado:

1. Seleccione la incidencia en la lista inferior.
2. Pulse el botón **"Marcar como Resuelta"** y damos a **"OK"**

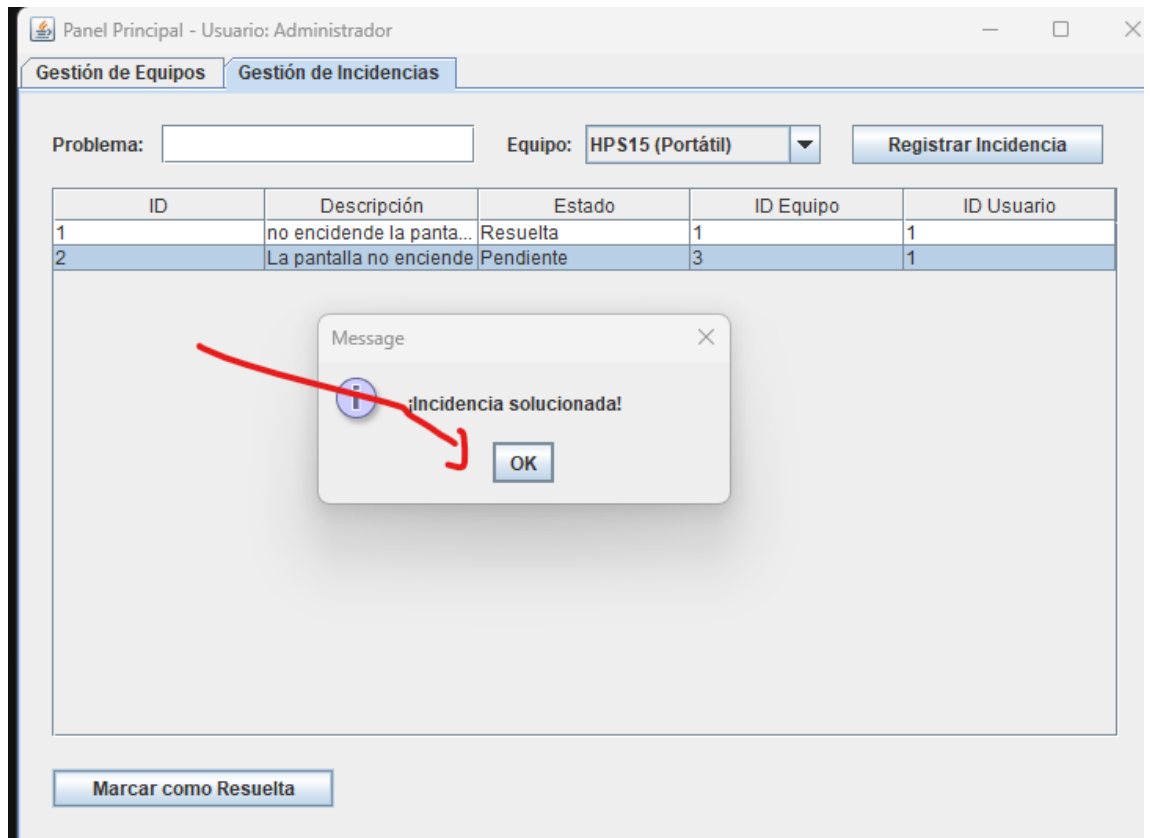


Panel Principal - Usuario: Administrador

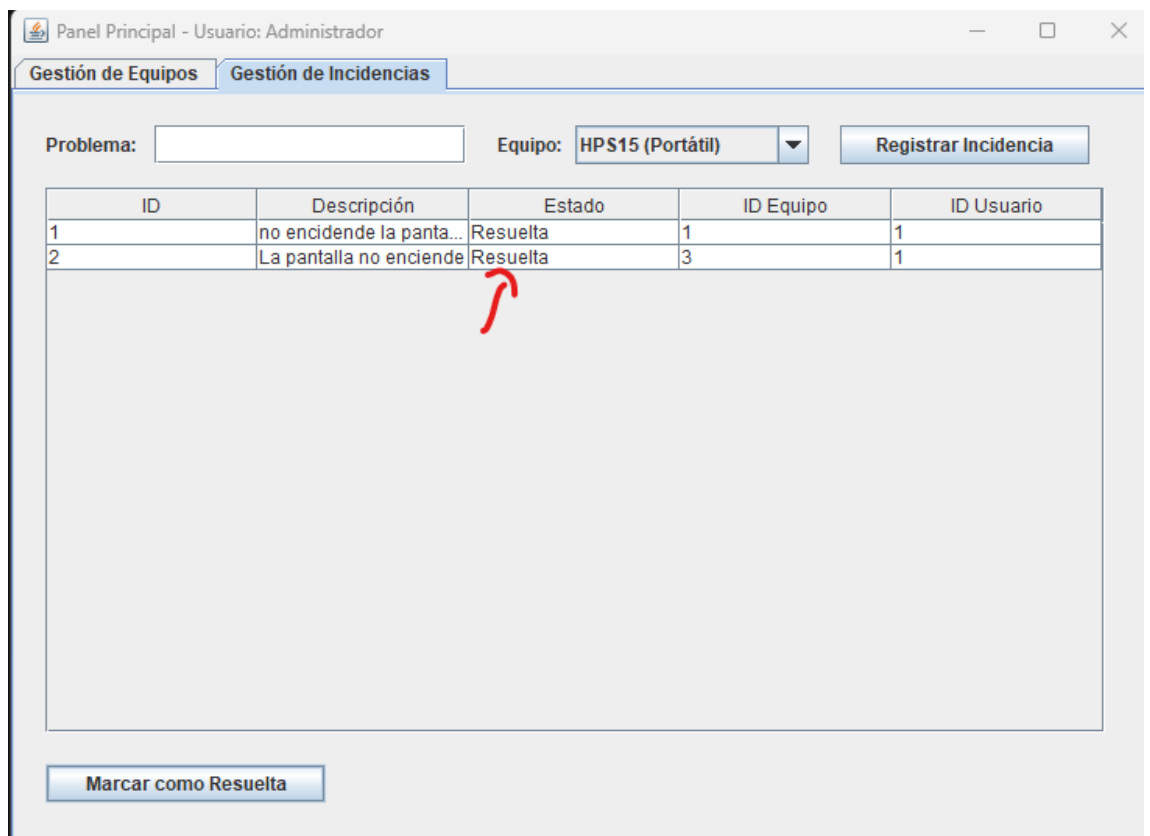
Gestión de Equipos Gestión de Incidencias

Problema: Equipo: HPS15 (Portátil)

ID	Descripción	Estado	ID Equipo	ID Usuario
1	no enciende la panta...	Resuelta	1	1
2	La pantalla no enciende	Pendiente	3	1



3. Observará que el estado cambia automáticamente en la tabla, permitiendo llevar un histórico de reparaciones finalizadas.



GLOSARIO DE TÉRMINOS

- **JDK (Java Development Kit):** Conjunto de herramientas software necesarias para desarrollar aplicaciones en lenguaje Java.
- **JRE (Java Runtime Environment):** Entorno necesario para que el ordenador pueda ejecutar la aplicación Java una vez terminada.
- **GUI (Graphical User Interface):** Interfaz gráfica de usuario. Es la parte visual del programa (ventanas, botones, tablas) con la que interactúa el técnico.
- **Swing:** Biblioteca oficial de Java que he utilizado para diseñar las ventanas y componentes visuales de la aplicación.
- **SQLite:** Sistema de gestión de bases de datos que no requiere un servidor externo, guardando toda la información en un solo archivo local.
- **JDBC (Java Database Connectivity):** Tecnología que permite al código Java "hablar" con la base de datos para leer o guardar información.
- **CRUD (Create, Read, Update, Delete):** Acrónimo que define las funciones básicas de cualquier gestión de datos: Crear, Leer, Actualizar y Borrar.
- **DAO (Data Access Object):** Patrón de diseño que separa la lógica de los datos de la interfaz visual, haciendo el código más ordenado y profesional.
- **Incidencia:** Reporte de un fallo, avería o error técnico asociado a un equipo específico del inventario.
- **Clave Foránea (Foreign Key):** Campo en una tabla de la base de datos que conecta un registro con otra tabla (por ejemplo, conecta una incidencia con un equipo específico).